

Alexander Urbaniak

(imię i nazwisko)

Kierunek: informatyka

Specjalność: informatyka stosowana

Specjalizacja: bazy danych i aplikacje internetowe

Numer albumu: 402199

**Porównanie skuteczności usług OCR – Azure AI
Document Intelligence vs. autorskie rozwiązania
EasyOCR i Tesseract**

(tytuł pracy dyplomowej)

Praca inżynierska

wykonana pod kierunkiem

dra Krzysztofa Podlaskiego

w Katedrze Systemów Inteligentnych

WFIS UŁ

Łódź, 2026

(rok kalendarzowy)

Spis treści

Streszczenie	3
Wstęp	4
Cel i zakres pracy	5
1. Charakterystyka technologii OCR i środowiska wytwarzania oprogramowania.....	6
1.1. Zasada działania optycznego rozpoznawania znaków (OCR).....	6
1.2. Przegląd wykorzystanych silników OCR.....	7
1.2.1. Tesseract OCR - metody klasyczne i LSTM	7
1.2.2. EasyOCR - wykorzystanie <i>deep learning</i>	8
1.2.3. Azure AI Document Intelligence - rozwiązania chmurowe	8
1.3. Środowisko implementacji.....	9
1.4. Metryki oceny jakości systemów OCR.....	10
2. Metodologia i implementacja projektu.....	13
2.1. Wybór i analiza metodologii	13
2.2. Zastosowane narzędzia i technologie.....	14
2.3. Architektura i projekt systemu	15
2.4. Etap implementacji	18
2.4.1. Moduł generatora danych testowych - ground truth.....	19
2.4.2. Główne API systemu i zarządzanie żadaniami	22
2.4.3. Moduł integracji silników OCR i post-processing.....	23
2.4.4. Automatyzacja testów i analiza wyników	27
2.5. Instrukcja uruchomienia i obsługi programu	28
3. Badania skuteczności i wydajności systemów OCR	31
3.1. Scenariusze i środowisko testowe	31
3.2. Wyniki testów wydajnościowych	33
3.3. Wyniki testów jakościowych.....	34
3.4. Dyskusja wyników	38
Wnioski i podsumowanie.....	40
Bibliografia	41
Spis rysunków.....	44
Spis wzorów	44
Spis kodów	44
Spis tabel	44
Abstract	45

Streszczenie

Celem niniejszej pracy inżynierskiej było zaprojektowanie, zaimplementowanie oraz przetestowanie systemu do automatycznego rozpoznawania i ekstrakcji danych z faktur VAT. Głównym zadaniem badawczym było obiektywne porównanie działania autorskiego mikroservisu OCR, wykorzystującego darmowe biblioteki EasyOCR oraz PyTesseract, z komercyjną usługą chmurową Azure AI Document Intelligence.

W ramach części praktycznej zrealizowano aplikację typu REST API w języku Python, która przetwarzała dostarczone obrazy dokumentów za pomocą wybranego silnika OCR, a następnie poddawała je procesowi strukturyzacji i zapisywała wyniki w relacyjnej bazie danych SQL. Aby zapewnić powtarzalne i wiarygodne środowisko testowe, opracowano dedykowany generator danych. Na podstawie 8 zróżnicowanych szablonów HTML wygenerowano zbiór syntetycznych faktur, które następnie przekształcono w dokumenty cyfrowe, płaskie skany oraz zniekształcone fotografie mobilne.

Zgodnie z założeniami, każda z badanych technologii została przetestowana na obszernym zestawie dokumentów. Wykorzystując dedykowane skrypty analityczne, przeprowadzono zautomatyzowany proces ewaluacji, który objął szczegółową analizę trzech kluczowych parametrów: skuteczności ekstrakcji i przypisywania danych do odpowiednich pól (m.in. numery faktur, daty, kontrahenci, pozycje towarowe, kwoty netto i brutto), dokładności rozpoznanego tekstu oraz czasu przetwarzania poszczególnych dokumentów.

Zrealizowany projekt oraz wygenerowane zestawienia wyników pozwoliły na kompleksowe porównanie badanych systemów.

Słowa kluczowe: optyczne rozpoznawanie znaków ekstrakcja danych z faktur
analiza układu przestrzennego architektura mikroservisowa
parsowanie i strukturyzacja danych

Keywords: optical character recognition invoice data extraction
spatial layout analysis microservice architecture
data parsing and structuring

Wstęp

W dobie cyfrowej transformacji przedsiębiorstw, efektywne zarządzanie dokumentacją stanowi jeden z kluczowych obszarów optymalizacji procesów biznesowych [1]. Tradycyjny obieg dokumentów papierowych, w szczególności faktur VAT, generuje znaczne koszty operacyjne związane z ich ręcznym przetwarzaniem, archiwizacją oraz ryzykiem popełnienia błędów podczas wprowadzania danych do systemów księgowych czy ERP (ang. *Enterprise Resource Planning*)¹. Odpowiedzią na te wyzwania jest technologia optycznego rozpoznawania znaków (ang. *OCR – Optical Character Recognition*), która umożliwia automatyczną konwersję obrazów dokumentów na format edytowalny i możliwy do przeszukania.

Rozwój algorytmów uczenia maszynowego (ang. *machine learning*)² oraz głębokich sieci neuronowych (ang. *deep learning*)³ w ostatnich latach znacząco wpłynął na ewolucję systemów OCR. Nowoczesne rozwiązania, takie jak modele wizyjno-językowe oparte na architekturze Transformer (np. LayoutLM) [2], nie ograniczają się już jedynie do prostego rozpoznawania liter, lecz potrafią analizować strukturę dokumentu, identyfikować tabele oraz rozumieć kontekst semantycznych informacji. Na rynku dostępne są zarówno zaawansowane usługi chmurowe oferowane przez gigantów technologicznych jak i biblioteki typu Open Source⁴, które pozwalają na budowę własnych, lokalnych rozwiązań.

Wybór odpowiedniej technologii do ekstrakcji danych z faktur jest decyzją niebanalną wymagającą uwzględnienia wielu czynników, takich jak dokładności rozpoznawania, koszt wdrożenia, wydajność oraz bezpieczeństwo danych. Niniejsza praca podejmuje próbę empirycznego porównania dwóch podejść do tego problemu: wykorzystania gotowej usługi chmurowej klasy Enterprise⁵ – Azure AI Document Intelligence, oraz budowy autorskiego mikroserwisu opartego na darmowych bibliotekach - Tesseract OCR, EasyOCR.

¹ Zintegrowany system informatyczny służący do planowania i zarządzania zasobami całego przedsiębiorstwa [36].

² Dziedzina sztucznej inteligencji, w której algorytmy samodzielnie uczą się wzorców z danych bez jawnego ich programowania [35].

³ Podkategoria uczenia maszynowego oparta na wielowarstwowych sieciach neuronowych, wysoce skuteczna m.in. w analizie obrazu [34].

⁴ Otwarte oprogramowanie, którego kod źródłowy jest publicznie dostępny do swobodnego użytku i modyfikacji [33].

⁵ Oprogramowanie dla dużych organizacji, charakteryzujące się najwyższą skalowalnością, bezpieczeństwem i niezawodnością [32].

Cel i zakres pracy

Głównym celem niniejszej pracy inżynierskiej jest zaprojektowanie, implementacja oraz przetestowanie systemu informatycznego służącego do automatycznej ekstrakcji danych z faktur VAT. System ten ma umożliwić obiektywne porównanie skuteczności i wydajności trzech wybranych technologii OCR w zróżnicowanych warunkach wejściowych - skany, zdjęcia, dokumenty cyfrowe.

Zakres pracy obejmuje realizację następujących zadań badawczych i projektowych:

1. Analiza i wybór technologii: Przegląd dostępnych rozwiązań OCR oraz wybór reprezentatywnych narzędzi do badań: Azure AI Document Intelligence - rozwiązanie chmurowe, Tesseract OCR - rozwiązanie klasyczne/LSTM oraz EasyOCR - rozwiązanie oparte na głębokim uczeniu.
2. Opracowanie generatora zbioru referencyjnego (ang. *ground truth*)⁶: Stworzenie oprogramowania w języku Python, które na podstawie przygotowanych szablonów HTML wygeneruje zbiór syntetycznych faktur o znanej strukturze. Zbiór ten posłuży jako wzorzec odniesienia do oceny poprawności działania algorytmów OCR.
3. Projekt i implementacja mikroserwisu OCR: Budowa aplikacji typu REST API⁷ przy użyciu frameworka FastAPI, integrującej wybrane silniki OCR oraz implementującej logikę parsowania i strukturyzacji danych (ang. *post-processing*).
4. Przeprowadzenie testów wydajnościowych i jakościowych: Przetestowanie zaimplementowanego rozwiązania na zestawie co najmniej 100 dokumentów dla każdej z trzech technologii. Badania obejmować będą analizę skuteczności ekstrakcji kluczowych pól m.in. numer faktury, daty, kwoty, pozycje Tab.ryczne oraz pomiar czasu przetwarzania.
5. Analiza wyników i wnioski: Opracowanie zestawienia porównawczego badanych technologii, wskazanie ich mocnych i słabych stron oraz sformułowanie rekomendacji dotyczących ich zastosowania w systemach komercyjnych.

Realizacja powyższych celów pozwoli na odpowiedź na pytanie badawcze: czy w typowych zastosowaniach biznesowych darmowe biblioteki OCR, przy odpowiednim przetworzeniu danych, mogą stanowić alternatywę dla płatnych usług chmurowych?

⁶ Zweryfikowany, bezbłędny zestaw danych referencyjnych, służący jako idealny wzorzec do oceny dokładności modeli sztucznej inteligencji [22].

⁷ Styl architektury oprogramowania wykorzystywany przy budowie usług sieciowych, oparty na bezstanowych żądaniach HTTP, gdzie zasoby serwera są identyfikowane za pomocą adresów URI [31].

1. Charakterystyka technologii OCR i środowiska wytwarzania oprogramowania

1.1. Zasada działania optycznego rozpoznawania znaków (OCR)

Technologia optycznego rozpoznawania znaków (ang. *Optical Character Recognition*, OCR) stanowi dziedzinę sztucznej inteligencji oraz przetwarzania obrazów, której zadaniem jest konwersja dwuwymiarowych reprezentacji tekstu - skanów, zdjęć, plików PDF na postać cyfrową, zakodowaną maszynowo np. w standardzie ASCII lub Unicode [3]. Proces ten nie ogranicza się jedynie do identyfikacji poszczególnych znaków, lecz obejmuje szereg złożonych etapów analizy, mających na celu odtworzenie struktury logicznej dokumentu.

Współczesne systemy OCR działają zazwyczaj w kilku fazach, które można podzielić na przetwarzanie wstępne (ang. *pre-processing*), segmentację, rozpoznawanie właściwe oraz przetwarzanie końcowe (ang. *post-processing*) [4].

Pierwszym etapem jest akwizycja i wstępne przetwarzanie obrazu. Surowe dane wejściowe, często pochodzące ze skanerów lub aparatów cyfrowych, obarczone są szumami, zniekształceniami perspektywy oraz nierównomiernym oświetleniem. Celem tej fazy jest maksymalne uwydawnienie treści tekstowej względem tła. Stosuje się tu techniki takie jak binaryzacja, czyli zamiana obrazu na czarno-biały, często z wykorzystaniem adaptacyjnego programowania, np. metodą Otsu [5], odszumianie np. filtry medianowe oraz korekcja nachylenia (ang. *deskewing*), która prostuje tekst względem osi poziomej.

Kolejnym kluczowym krokiem jest analiza układu strony i segmentacja (ang. *layout analysis*). Algorytm musi odróżnić bloki tekstu od elementów graficznych, takich jak logotypy, linie czy tabele. W przypadku faktur jest to szczególnie istotne, gdyż dane rozmieszczone są w nieregularnych kolumnach. Segmentacja następuje zazwyczaj hierarchicznie: obraz dzielony jest na akapity, następnie na linie tekstu, wyrazy, a ostatecznie na pojedyncze znaki lub ich fragmenty.

Właściwe rozpoznawanie znaków realizowane jest obecnie dwiema głównymi metodami [6]:

1. Metody oparte na wzorcach i cechach (ang. *feature extraction*) – klasyczne podejście, w którym algorytm analizuje geometrię znaku (linie, łuki, pętle) i porównuje ją z bazą wzorców.
2. Metody oparte na sieciach neuronowych – nowoczesne podejście wykorzystujące głębokie uczenie. Historycznie stosowano tutaj konwolucyjne sieci neuronowe (CNN) oraz sieci rekurencyjne takie jak LSTM (Long Short-Term Memory). Współcześnie jednak następuje wyraźny zwrot ku architekturze Transformer

i modelom wizyjno-językowym takim jak LayoutLM. Wykorzystują one mechanizm uwagi, co pozwala im nie tylko na skuteczne rozpoznawanie samych znaków, ale również na równoległą analizę układu przestrzennego i kontekstu semantycznego całego dokumentu.

Ostatnim etapem jest przetwarzanie końcowe, które ma na celu korekcję błędów rozpoznawania przy użyciu słowników językowych oraz modeli statycznych np. n-gramów⁸. W kontekście dokumentów finansowych, takich jak faktury, etap ten obejmuje również walidację danych (np. sprawdzenie sumy kontrolnej numeru NIP czy poprawności formatu daty), co w niniejszym projekcie realizowane jest przez dedykowane moduły parsujące.

1.2. Przegląd wykorzystanych silników OCR

W ramach projektu do ekstrakcji danych tekstowych z obrazów faktur wybrano trzy odmienne pod względem architektury i podejścia do problemu narzędzia. Pozwoli to na przekrojową analizę skuteczności dostępnych na rynku rozwiązań – od klasycznych systemów open source, przez nowoczesne biblioteki głębokiego uczenia, aż po zaawansowane usługi chmurowe klasy Enterprise.

1.2.1. Tesseract OCR - metody klasyczne i LSTM

Tesseract OCR to jeden z najstarszych i najbardziej rozpoznawalnych silników optycznego rozpoznawania znaków, rozwijany pierwotnie przez Hewlett-Packard, a od 2006 roku wspierany przez Google. Jest to rozwiązanie typu open source, udostępniane na licencji Apache 2.0 [7].

Kluczowym momentem w rozwoju Tesseracta było wprowadzenie w wersji 4.0 [8] nowego silnika opartego na rekurencyjnych sieciach neuronowych typu LSTM (Long Short-Term Memory) w 2018 r. Wcześniejsze wersje opierały się głównie na dopasowywaniu wzorców i analizie kształtów, np. wykrywaniu linii bazowych czy wysokości x-height. Model LSTM pozwala na analizę całych sekwencji znaków - linii tekstu, jednocześnie, zamiast rozpoznawania każdej litery z osobna. Dzięki temu system „uczy się” kontekstu językowego, co znacząco poprawia skuteczność w przypadku dokumentów o niskiej jakości lub nietypowych czcionkach [9].

W niniejszym projekcie wykorzystano bibliotekę pytesseract⁹ – nakładkę (ang. *wrapper*) dla języka Python, który komunikuje się z binarnym plikiem wykonywalnym Tesseracta,

⁸ W przetwarzaniu języka naturalnego n-gram to ciągła sekwencja n elementów (np. liter, sylab lub słów) z danego tekstu, używana w modelach probabilistycznych do przewidywania następnego elementu w sekwencji [30].

⁹ Nakładka umożliwiająca jej wykorzystanie w języku Python poprzez uproszczony interfejs programistyczny [29].

przekazując obraz i parametry konfiguracyjne np. język polski, tryb segmentacji strony PSM.

1.2.2. EasyOCR - wykorzystanie *deep learning*

EasyOCR to nowoczesna biblioteka języka Python, stworzona przez Jaided AI, która od podstaw została zaprojektowana z wykorzystaniem techniki głębokiego uczenia w oparciu o framework PyTorch. W odróżnieniu od Tesseracta, EasyOCR nie jest monolitycznym silnikiem, lecz składa się z potoku przetwarzania (ang. *pipeline*) łączącego kilka wyspecjalizowanych modeli sieci neuronowych [10].

Architektura EasyOCR opiera się na dwóch głównych komponentach:

1. Detekcja tekstu (ang. *text detection*):
Wykorzystuje model CRAFT (Character Region Awareness for Text Detection). Jego zadaniem jest zlokalizowanie obszarów na obrazie, które z wysokim prawdopodobieństwem zawierają tekst, niezależnie od ich orientacji, krzywizny czy tła.
2. Rozpoznawanie tekstu (ang. *text recognition*):
Wyciągnięte fragmenty obrazu trafiają do modelu CRNN (Convolutional Recurrent Neural Network), który łączy konwolucyjne sieci neuronowe¹⁰ (CNN) do ekstrakcji cech wizualnych z rekurencyjnymi sieciami¹¹ (RNN/LSTM) do sekwencyjnego odczytu znaków. Na końcu stosowany jest algorytm CTC¹² (Connectionist Temporal Classification) do dekodowania wyników.

Dzięki takiej budowie EasyOCR charakteryzuje się wysoką skutecznością w trudnych warunkach np. zdjęcia pod kątem, nierównomierne oświetlenie, co czyni go idealnym kandydatem do testów na zdjęciach faktur.

1.2.3. Azure AI Document Intelligence - rozwiązania chmurowe

Azure AI Document Intelligence (dawniej znany jako Azure Form Recognizer) to usługa chmurowa typu SaaS (Software as a Service)¹³ oferowana przez firmę Microsoft. Jest to rozwiązanie klasy Enterprise, które wykorzystuje zaawansowane modele uczenia maszynowego trenowane na ogromnych zbiorach dokumentów biznesowych.

¹⁰ Rodzaj sieci neuronowej zaprojektowanej do automatycznego uczenia się hierarchicznych reprezentacji danych poprzez zastosowanie operacji konwolucji, szczególnie skuteczny w analizie obrazów i danych o strukturze przestrzennej [34].

¹¹ Modele przeznaczone do przetwarzania danych sekwencyjnych, wskazując jednocześnie na ich ograniczenia związane z zanikiem gradientu, które zostały częściowo rozwiązane poprzez architekturę LSTM [34].

¹² Funkcja kosztu stosowana w modelach sekwencyjnych, umożliwiającą efektywne uczenie bez jawnego wyrównania danych wejściowych i wyjściowych poprzez sumowanie po wszystkich możliwych dopasowaniach sekwencji [28].

¹³ Model, w którym użytkownik korzysta z aplikacji działających w infrastrukturze chmurowej dostawcy, dostępnych przez sieć (np. przeglądarkę), bez zarządzania systemem operacyjnym czy infrastrukturą [26].

W przeciwieństwie do bibliotek OCR, które zwracają jedynie „surowy” tekst, usługa Azure oferuje tzw. „*modele pre-build*” (wstępnie wytrenowane), w tym dedykowany do faktur „*invoice model*”. Potrafi on nie tylko rozpoznawać znaki, ale również automatycznie zidentyfikować i wyodrębnić kluczowe pary klucz-wartość, takie jak:

- Dane sprzedawcy i nabywcy (NIP, nazwa, adres),
- Daty wystawienia i płatności,
- Wartości kwotowe wraz z walutą,
- Pozycje Tab.ryczne (wiersze produktów/usług).

Komunikacja z usługą odbywa się poprzez interfejs REST API, gdzie aplikacja kliencka przesyła obraz dokumentu, a w odpowiedzi otrzymuje ustrukturyzowany format JSON zawierający zarówno rozpoznany tekst, jak i jego semantyczną interpretację wraz ze współrzędnymi (ang. *bounding boxes*) i poziomem pewności (ang. *confidence score*).

1.3. Środowisko implementacji

Wybór odpowiedniego środowiska programistycznego oraz narzędzi pomocniczych ma kluczowe znaczenie dla efektywności procesu tworzenia oprogramowania, a także późniejszej wydajności i skalowalności systemu. W niniejszej pracy zdecydowano się na wykorzystanie języka Python w wersji 3.13 jako głównego języka implementacji. Decyzja ta podyktowana była przede wszystkim bogatym ekosystemem bibliotek dedykowanych przetwarzaniu danych, uczeniu maszynowemu (ML) oraz wizji komputerowej (CV), a także łatwością integracji z zewnętrznymi interfejsami API [11].

Do budowy warstwy serwerowej aplikacji wykorzystano nowoczesny framework webowy FastAPI. W odróżnieniu od starszych rozwiązań, takich jak Flask czy Django, FastAPI opiera się na standardzie ASGI (Asynchronous Server Gateway Interface) [12] oraz wykorzystuje wbudowaną obsługę typowania statycznego (Type Hints) języka Python. Dzięki temu zapewnia nie tylko wysoką wydajność (porównywalną z Node.js [13] czy Go [14]), ale również automatyczną generację interaktywnej dokumentacji API (Swagger UI), co znacząco przyspiesza proces testowania punktów końcowych, tzw. „*endpointów*”.

W warstwie persystencji danych zastosowano lekki silnik bazodanowy SQLite, który w zupełności wystarcza do przechowywania wyników testów oraz metadanych faktur w środowisku deweloperskim [15]. Komunikacja z bazą odbywa się ze pośrednictwem biblioteki SQLAlchemy – popularnego w świecie Pythona narzędzia typu ORM (Object-Relational Mapping) [16]. Pozwala ono na mapowanie klas i obiektów pythonowych na tabele relacyjnej bazy danych, eliminując konieczność ręcznego pisania zapytań SQL i zwiększając przenośność kodu.

Do przetwarzania obrazów i generowania dokumentów PDF wykorzystano zestaw bibliotek pomocniczych:

- OpenCV (cv2) oraz NumPy: Biblioteka OpenCV odpowiada za zaawansowane przetwarzanie obrazu - binaryzacja, odszumianie, natomiast NumPy stanowi fundament obliczeniowy, umożliwiając reprezentację obrazów jako wielowymiarowych macierzy numerycznych (`numpy.ndarray`), na których operują algorytmy wizyjne.
- Pillow (PIL): Biblioteka do podstawowych operacji na grafikach rastrowych, służąca m.in. do konwersji formatów i kadrowania.
- PDFKit oraz wkhtmltopdf: Zestaw narzędzi do generowania dokumentów PDF. PDFKit pełni rolę interfejsu (tzw. „wrappera”) dla języka Python, sterującego zewnętrznym silnikiem renderującym wkhtmltopdf, który konwertuje kod HTML/CSS na format PDF.
- Pdf2image oraz Poppler: Biblioteka pdf2image służy do rastryzacji dokumentów PDF na obrazy. Wymaga ona obecności w systemie zewnętrznego narzędzia Poppler, które odpowiada za niskopoziomowe renderowanie stron dokumentu.
- Faker: Biblioteka wykorzystana w module generatora danych do tworzenia realistycznych, lecz syntetycznych danych testowych (nazwy firm, adresy, numery NIP), co zapewnia zgodność z RODO.
- Python-dotenv: Narzędzie do zarządzania zmiennymi środowiskowymi, pozwalające na bezpieczne przechowywanie kluczy API np. do usługi Azure oraz konfiguracji ścieżek systemowych poza kodem źródłowym aplikacji.

Całość projektu została zorganizowana w formie modułowej, z wykorzystaniem wirtualnego środowiska (ang. *venv*) do izolacji zależności, co zapewnia powtarzalność wyników i łatwość uruchomienia na innej maszynie.

1.4. Metryki oceny jakości systemów OCR

Zarządzanie jakością ekstrakcji danych z wykorzystaniem systemów klasy OCR wymaga zastosowania precyzyjnych i obiektywnych metryk, które pozwolą na ilościowe określenie skuteczności algorytmów. W odróżnieniu od klasycznego rozpoznawania bloków tekstu ciągłego, gdzie powszechnie stosuje się miary takie jak CER (Character Error Rate) [17] czy WER (Word Error Rate) [18], analiza dokumentów finansowych o wysoce ustrukturyzowanej formie (faktury VAT) narzuca konieczność ewaluacji na poziomie semantycznym. Oznacza to, że z perspektywy systemu biznesowego błąd w jednym znaku (np. cyfrze w kwocie netto lub w numerze NIP) całkowicie dyskwalifikuje poprawność odczytu całego pola.

Mając to na uwadze, w procesie badawczym niniejszej pracy zrezygnowano z klasycznego zliczania błędów znakowych za rzecz ewaluacji o dokładność dopasowywania konkretnych encji danych do wartości referencyjnych (tzw. zbioru *Ground Truth*).

Do oceny zaimplementowanego mikroserwisu wykorzystano następujący zbiór wskaźników jakościowych:

Dokładność rozpoznawania pojedynczych atrybutów (field-level accuracy)

Podstawową miarą oceny skuteczności algorytmów w przypadku pól statycznych (np. data wystawienia, NIP sprzedawcy, numer faktury) jest wskaźnik dokładności binarnie kategoryzujący wyniki jako poprawny lub błędny. [19] Miara ta definiowana jest jako stosunek liczby pól zidentyfikowanych w pełni prawidłowo (zgodnych z wartością referencyjną co do znaku, po procesie normalizacji uwzględniającej usuwanie zbędnych białych znaków) do całkowitej liczby badanych atrybutów danego typu. Dodatkowo analizie poddawane są odczyty puste (tzw. zjawisko „*missing data*”), wskazujące na niezgodność danego silnika OCR do jakiegokolwiek lokalizacji bloku testowego.

Ewaluacja struktur Tab.rycznych (precyzja, czułość i F1-score)

Ekstrakcja list asortymentowych (tj. pozycji na fakturze) stanowi proces wieloetapowy, polegający nie tylko na odczytaniu tekstu, ale również na właściwej segmentacji wierszy. Do oceny tego elementu zaadaptowano klasyczne metryki znane z dziedziny systemów wyszukiwania informacji (ang. *information retrieval*) [20]. Oparto je o macierz błędów (tzw. „*confusion matrix*”), wyróżniając:

- True Positives (TP) – prawidłowo zidentyfikowane i zmapowane pozycje towarowe.
- False Positives (FP) – pozycje zwrócone przez silnik OCR, które w rzeczywistości nie znajdowały się na dokumencie (wyniki błędnej segmentacji szumu tła lub innych bloków tekstu jako tabeli).
- False Negatives (FN) – pozycje fizycznie obecne na fakturze referencyjnej, które zostały pominięte przez mechanizm parsowania.

Na ich podstawie w module testowym wyliczane są trzy parametry:

1. Precyzja (ang. *precision*) – określająca odsetek poprawnie rozpoznanych pozycji względem wszystkich pozycji wyekstrahowanych przez dany silnik, którą definiuje się wzorem 1.

$$\frac{TP}{(TP + FP)} \quad (1)$$

Równanie 1. Precyzja.

2. Czułość (ang. *recall*) – definiująca zdolność algorytmu do odnalezienia wszystkich rzeczywiście istniejących pozycji zdefiniowanych w *ground truth*. Wyliczana jest ze wzoru 2.

$$\frac{TP}{(TP + FN)} \quad (2)$$

Równanie 2. Czułość.

3. F1-score – średnia harmoniczna precyzji i czułości, stanowiąca najbardziej wyważoną i miarodajną metrykę ogólnej zdolności silnika do analizy Tab.rycznej, wyrażonej wzorem 3.

$$\frac{2 \cdot (Precision \cdot Recall)}{(Precision + Recall)} \quad (3)$$

Równanie 3. F1-score.

Błąd estymacji wartości numerycznych (MAE i MRE)

Z biznesowego oraz prawnego punktu widzenia, proces automatyzacji przetwarzania faktur wymaga zerowej tolerancji dla błędów w polach numerycznych, takich jak kwota netto, podatek VAT czy wartość brutto. Wartości finansowe muszą być ekstrahowane ze stuprocentową bezbłędnością. W docelowych systemach produkcyjnych ocena takich pól sprowadza się zatem do rygorystycznej weryfikacji, w której dopuszczalny jest wyłącznie całkowity sukces.

Niemniej jednak na etapie badawczym i podczas empirycznego porównywania silników OCR, kategoryzacja binarna prawda/fałsz często nie dostarcza pełnego obrazu sytuacji i ukrywa informacje o naturze popełnianych błędów. Algorytm mógł bowiem prawidłowo odczytać większość cyfr w kwocie, myląc się jedynie o rząd wielkości z powodu błędnego odczytu separatora dziesiętnego, na przykład pomylenia przecinka z kropką. Wiedza o tym, czy dany model generuje całkowicie losowe liczby, czy też popełnia drobne, powtarzalne błędy możliwe do łatwej korekty w fazie przetwarzania końcowego, jest kluczowa dla oceny jego przydatności. W celu rzetelnego zbadania odchyłeń numerycznych w pracy zastosowano miary:

- MAE (Mean Absolute Error) – średni błąd bezwzględny, informujący o średniej bezwzględnej różnicy pomiędzy kwotą zwróconą przez parser a kwotą bazową.
- MRE (Mean Relative Error) – średni błąd względny, wyrażający wielkość błędu w odniesieniu do skali pierwotnej wartości liczbowej, co pozwala zniwelować wpływ różnic w rzędach wielkości poszczególnych kwot (np. błąd o 10 PLN przy kwocie 100 PLN ma inną wagę niż przy kwocie 10000 PLN).

Ostatnim badanym aspektem jest wydajność czasowa systemu. Metryka ta zbiera rzeczywisty czas odpowiedzi poszczególnych adaptacji silników mierzony w milisekundach, pozwalając na wyznaczenie wartości średniej, minimalnej oraz maksymalnej dla każdej przetwarzanej próbki, co bezpośrednio determinuje zasadność wykorzystywania danej technologii w systemach o rygorystycznych wymagach produkcji (tzw. SLA) [21].

2. Metodologia i implementacja projektu

2.1. Wybór i analiza metodologii

Kluczowym wyzwaniem w badaniach nad skutecznością systemów OCR jest dostęp do wiarygodnego i reprezentowanego zbioru danych testowych, zwanego w literaturze [22] przedmiotu „ground truth” (GT). W kontekście analizy dokumentów finansowych, problem ten jest szczególnie istotny z uwagi na ochronę danych osobowych (RODO) oraz tajemnicę przedsiębiorstwa, co uniemożliwia wykorzystywanie rzeczywistych faktur w pracach badawczych bez kosztownej anonimizacji.

Wobec powyższych ograniczeń, w niniejszej pracy przyjęto metodologię hybrydową, łączącą generowanie syntetycznych danych tekstowych z fizycznym procesem digitalizacji. Podejście to polega na stworzeniu oprogramowania, które automatycznie generuje teść faktur na podstawie zdefiniowanych szablonów HTML, wypełniając je losowymi, lecz semantycznie poprawnymi danymi (nazwa firm, kwoty, daty).

Główną zaletą tej metody jest uzyskanie pełnej kontroli nad zawartością dokumentu. Ponieważ system generuje fakturę od podstaw, posiada on dokładną wiedzę o tym, jakie wartości powinny zostać odczytane z poszczególnych pól (np. „numer faktury”, „kwota netto”), co eliminuje konieczność ręcznego opisywania (ang. *labelingu*) setek obrazów.

Metodologia badawcza zakłada przeprowadzenie eksperymentu w trzech etapach:

1. Przygotowanie fizycznego zbioru testowego:
 - Generacja: Wygenerowanie zestawu faktur w formacie PDF.
 - Wydruk: Fizyczny wydruk dokumentów na papierze w standardowym formacie A4.
 - Digitalizacja (Skanowanie): Ponowne wprowadzenie dokumentów do systemu poprzez użycie skanera biurowego (symulacja typowego obiegu dokumentów w firmie).
 - Fotografowanie: Wykonanie zdjęć wydrukowanych dokumentów przy użyciu smartfonu w statycznych, kontrolowanych warunkach oświetleniowych, co pozwala wyeliminować wpływ przypadkowych czynników zewnętrznych (cienie, odbłaski) na wyniki eksperymentu. Celem tego etapu jest zbadanie wpływu samej techniki akwizycji obrazu (zniekształcenia obiektywu, szum matrycy aparatu) na jakość rozpoznawania tekstu.
2. Przetwarzanie OCR: Przepuszczanie każdego obrazu (zarówno cyfrowego oryginału, jak i fizycznych kopii – skanu i zdjęcia) przez trzy silniki (Tesseract, EasyOCR, Azure).

3. Ewaluacja automatyczna: Porównanie wyników rozpoznawania tekstu z pierwotnymi danymi wzorcowymi (GT) przy użyciu zdefiniowanych metryk jakościowych.

Tak skonstruowany proces badawczy pozwala na badania wpływu rzeczywistych zakłóceń fizycznych (zagięcia papieru, faktura druku, refleksy świetlne, kąt nachylenia aparatu) na skuteczność algorytmów OCR, co stanowi istotną wartość dodaną względem symulacji cyfrowych.

2.2. Zastosowane narzędzia i technologie

Implementacja systemu wymagała integracji wysokopoziomowego języka Python z niskopoziomowymi narzędziami systemowymi oraz bibliotekami obliczeniowymi. Architektura rozwiązania opiera się na kilku kluczowych grupach narzędzi, które odpowiadają za generowanie danych, przetwarzanie obrazu oraz komunikację z silnikami OCR.

Zależności systemowe i silniki zewnętrzne.

Istotnym aspektem implementacji jest wykorzystanie zewnętrznego oprogramowania (tzw. „binariów systemowych”), bez którego biblioteki języka Python nie mogłyby funkcjonować. W projekcie wykorzystano następujące komponenty zainstalowane w systemie operacyjnym Windows:

- **Poppler:** Jest to biblioteka systemowa służąca do renderowania plików PDF. W projekcie stanowi ona niezbędny fundament dla biblioteki pythonowej pdf2image. Ponieważ Python nie posiada wbudowanego mechanizmu rasteryzacji dokumentów PDF do formatów graficznych (JPG/PNG), wykorzystano narzędzia linii poleceń Popplera (wskazane w konfiguracji aplikacji ścieżką: `C:\Program Files\poppler-24.02.0\Library\bin`).
- **Wkhtmltopdf:** Kolejnym niezbędnym programem zewnętrznym, zintegrowanym w procesie przygotowywania danych testowych, jest otwarte oprogramowanie wykorzystujące silnik renderujący QT WebKit w celu konwersji kodu HTML (wraz z arkuszami stylów CSS) do bezstratnego formatu PDF. W ramach projektu konieczne było bezpośrednie wpięcie pliku binarnego tego oprogramowania w architekturę skryptu generującego (poprzez wskazanie pliku wykonywalnego: `C:\Program Files\wkhtmltopdf\bin\wkhtmltopdf.exe`). Obiekt konfiguracyjny biblioteki pdfkit komunikuje się z tym procesem za pomocą standardowych protokołów wejścia/wyjścia (I/O), co pozwala na dynamiczną transformację wyrenderowanych szablonów faktur VAT do postaci cyfrowych dokumentów o ustandaryzowanych wymiarach i stałej rozdzielczości DPI.
- **Tesseract-OCR:** Trzecim kluczowym komponentem systemowym jest silnik OCR rozwijany przez Google. W przeciwieństwie od rozwiązań chmurowych

(np. Azure), które wykonują obliczenia na zewnętrznych serwerach, architektura silnika Tesseract narzuca konieczność uruchomienia analizy bezpośrednio na procesorze hosta. Ponieważ biblioteka pytesseract pełni jedynie rolę nakładki (tzw. „wrappera”), konieczne było zainstalowanie skompilowanych plików binarnych Tesseracta w systemie operacyjnym oraz ręczne skierowanie interpretera Pythona na ścieżkę uruchomieniową:

```
C:\Program Files\Tesseract-OCR\tesseract.exe.
```

Takie podejście umożliwiło wywołanie analizy optycznej z poziomu serwera FastApi, przekazując strumień bajtów przesyłanych obrazów bezpośrednio do procesu analizującego, bez konieczności ich uprzedniego zapisywania na dysku twardym.

Warto również zauważyć, że stabilność wyżej wymienionych procesów w dużej mierze zależy od prawidłowego odizolowania środowiska uruchomieniowego. Aby zagwarantować spójność wersji pakietów oraz poprawność działania interfejsów systemowych, w projekcie posłużono się mechanizmem izolacji środowiska wirtualnego (katalog `.venv`). Powiązano go z plikiem definicyjnym `requirements.txt`, który kategoryzuje i narzuca konkretne wersje zależności (takich jak Pillow, OpenCV-python czy moduł uczenia głębokiego Torch wykorzystywane przez silnik EasyOCR). Dzięki precyzyjnemu powiązaniu wysokopoziomowych bibliotek ekosystemu Python z odpowiednimi, natywnymi plikami wykonywalnymi, uzyskano stabilną architekturę zdolną do masowego, asynchronicznego procesowania plików graficznych.

2.3. Architektura i projekt systemu

Zaprojektowany system został zrealizowany w architekturze modularnego mikroservisu, udostępniającego swoje funkcjonalności za pośrednictwem interfejsu programistycznego (API) typu REST. Wybór takiego podejścia architektonicznego gwarantuje wysoką skalowalność rozwiązania oraz ścisłą separację logiki biznesowej od warstwy prezentacji czy zewnętrznych narzędzi testowych [23]. Głównym szkieletem serwera aplikacji jest nowoczesny framework webowy FastApi, który dzięki natywnemu wsparciu dla synchroniczności (mechanizm „`async/await`”) idealnie sprawdza się w operacjach o dużym obciążeniu wyjścia/wejścia (I/O), takich jak wielowątkowe przesyłanie plików graficznych. Struktura projektu została zaprezentowana na rys. 1.


```

INZ/
├── .venv/                # wirtualne środowisko
├── data/                 # dane wejściowe (contractors.json, items.json)
├── layouts/              # szablony HTML faktur
├── out/
│   ├── pdf/              # wygenerowane faktury PDF
│   ├── png_clean/        # wygenerowane obrazy faktur
│   ├── jpg_photo/        # zdjęcia faktur
│   ├── png_scan/         # skany faktur
│   └── results/          # wyniki testów OCR
├── app/                  # główny kod mikroserwisu OCR
│   ├── main.py           # FastAPI - endpointy REST
│   ├── ocr/
│   │   ├── easyocr_adapter.py    # adapter EasyOCR
│   │   ├── pytesseract_adapter.py # adapter PyTesseract
│   │   └── azure_adapter.py      # adapter Azure
│   ├── parsers/
│   │   ├── table_parser.py # parser tabel
│   │   ├── text_parser.py  # parser tekstu
│   │   └── invoice_parser.py # parsery tekstu faktur
│   └── db/
│       ├── database.py        # konfiguracja SQLAlchemy
│       ├── models.py          # wszystkie modele (GT + OCR)
│       ├── crud_ground_truth.py # CRUD dla generatora
│       ├── crud_ocr.py         # CRUD dla OCR
│       └── init_db.py          # automatyczne tworzenie tabel
├── scripts/
│   ├── client_submit.py # pojedynczy test OCR
│   ├── batch_test.py    # testy wsadowe
│   ├── generator.py      # generowanie faktur i danych GT
│   └── benchmark.py      # analiza wyników OCR
├── requirements.txt      # zależności projektu
└── invoices.db           # baza danych SQLite

```

Rys. 1. Struktura projektu.

Struktura logiczna i przepływ danych

Kod źródłowy aplikacji (`app/`) został podzielony na odrębne pakiety, realizujące ściśle określone odpowiedzialności, zgodnie z paradygmatem SoC (Separation of Concerns)¹⁴:

1. Moduł główny (`main.py`) – odpowiada za deklarację endpointów API (w tym głównego punktu dostępowego `/ocr`), walidację parametrów wejściowych (wybór silnika OCR, tryb testowy, układ graficzny faktury) oraz przekazywanie bajtów obrazu do odpowiednich serwisów parsujących.
2. Adapter OCR (`app/ocr/`) – warstwa abstrakcji (wzorec projektowy Adapter), która unifikuje sposób komunikacji z różnymi silnikami. Wyróżniono w niej implementacje dla EasyOCR, Tesseract oraz Azure AI. Każdy adapter ma za zadanie przyjąć obraz i zwrócić zunifikowaną strukturę danych, obejmującą zarówno pełny tekst (raw text), jak i koordynaty przestrzenne poszczególnych słów (bounding boxes).
3. Parsery semantyczne (`app/parsers/`) – kluczowy element logiki biznesowej. Surowe dane tekstowe zwracane przez adaptery są tutaj poddawane obróbce z wykorzystaniem wyrażeń regularnych (tzw. „RegEx”) i heurystyk przestrzennych. Wdrożono osobne parsery do analizy metadanych dokumentu (`text_parser.py`) oraz wysoce złożony parser tabel (`table_parser.py`), którego zadaniem jest rekonstrukcja relacji przestrzennych wierszy i kolumn asortymentowych faktury.

Modelowanie i utrwalanie danych

Aby umożliwić rzetelną weryfikację jakości zaimplementowanych algorytmów, system musiał zostać wyposażony w mechanizm trwałego przechowywania danych (tzw. „persystencji”). Zastosowano lekką, plikową relacyjną bazę danych SQLite (`invoices.db`), z którą aplikacja komunikuje się poprzez narzędzie do mapowania obiektowo-relacyjnego (ORM)¹⁵– SQLAlchemy.

Model bazy danych (`app/db/models.py`) został zaprojektowany z myślą ewaluacji porównawczej i opiera się na dwóch lustrzanych hierarchiach tabel, oddzielających dane wzorcowe od wyników zwróconych przez system. W strukturze bazy danych wyróżniono następujące encje:

- Tab. `ground_truth_invoices` oraz `ground_truth_items` – przechowują bezbłędne, referencyjne dane wygenerowane w procesie tworzenia sztucznych faktur (ang. *ground truth*). Każda faktura posiada przypisane do siebie pozycje

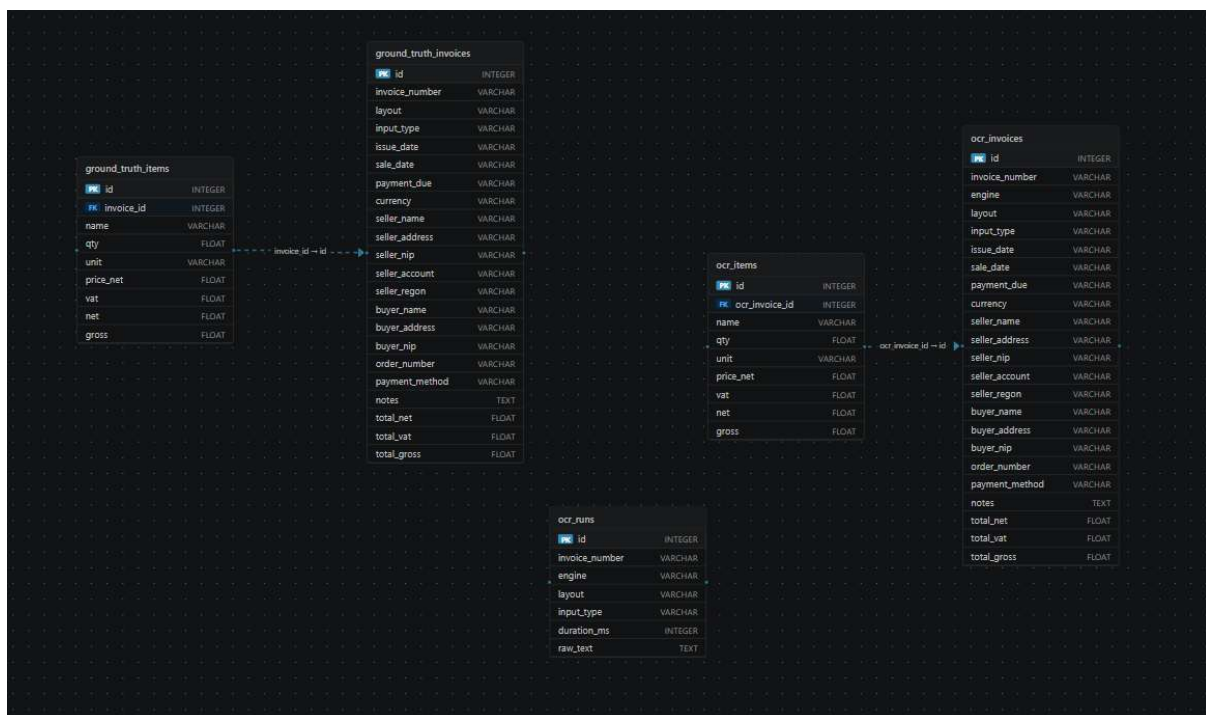
¹⁴ Zasada projektowania oprogramowania polegająca na podziale systemu na niezależne części, z których każda odpowiada za inny aspekt działania [27].

¹⁵ Podejście polegające na odwzorowaniu elementów modelu obiektowego (klas, obiektów i relacji) na odpowiadające im struktury w relacyjnej bazie danych, w celu ułatwienia komunikacji pomiędzy aplikacją a bazą danych [37].

asortymentowe, łącząc się z nimi relacją jeden-do-wielu (klucz obcy `invoice_id`).

- Tab. `ocr_invoices` oraz `ocr_items` – tabele przechowujące encje zwrócone w drodze parsowania obrazu przez jeden z trzech badanych silników. Poza polami finansowymi (NIP, nazwy firmy, daty, kwoty netto i brutto), w tabeli głównej przechowywana jest informacja o użytym silniku (`engine`), zniekształceniu obrazu (`input_type` – np. skan, zdjęcie, plik czysty) oraz rozpoznanym wzorcu wizualnym dokumentu (`layout`).
- Tab. `ocr_runs` – służy jako dziennik operacyjny (`log`). Agreguje informacje analityczne, w tym przede wszystkim surowy tekst zwrócony z danego modułu oraz dokładny czas wykonania operacji wyrażony w milisekundach (`duration_ms`), co pozwala na weryfikację wydajności badanych rozwiązań.

Taka architektura, której diagram został zaprezentowany na rys. 2, pozwala zautomatyzowanym skryptom (jak omówiony z dalszej części pracy mechanizm `batch_test`) na cykliczne wprowadzanie dokumentów do systemu, a następnie jednoznaczne i powtarzalne porównywanie rekordów sparowanych tabel (`ground_truth` kontra `ocr`) w celu wyliczenia wskaźników precyzji i błędów MAE.



Rys. 2. Diagram bazy danych.

2.4. Etap implementacji

Niniejszy podrozdział zawiera szczegółowy opis implementacji najważniejszych modułów systemu, począwszy od generowania zbioru testowego, przez warstwę abstrakcji silników OCR, aż po algorytmy ekstrakcyjne i proces testów wsadowych.

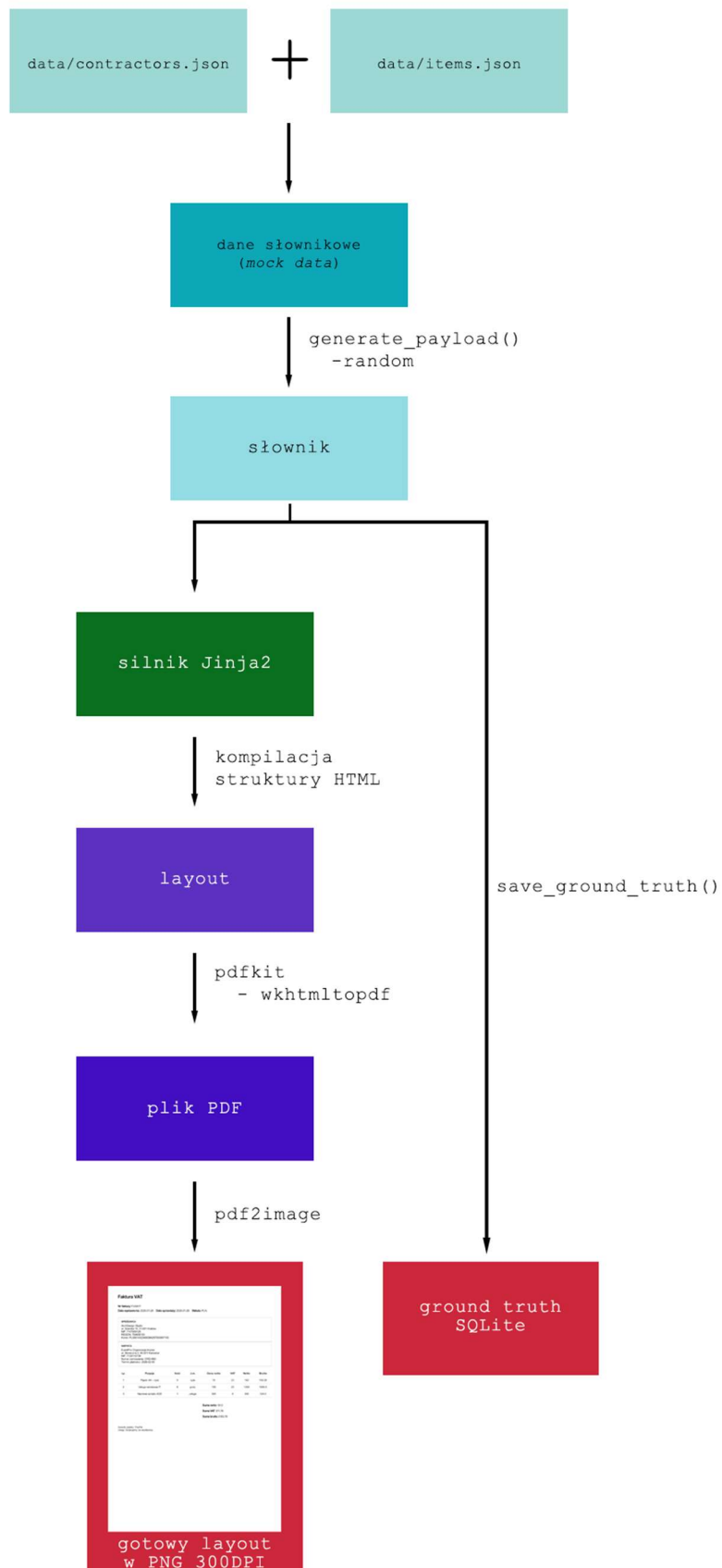
2.4.1. Moduł generatora danych testowych - ground truth

Aby przeprowadzić wiarygodne badanie porównawcze, konieczne było utworzenie kontrolnego zbioru danych. Zamiast wykorzystywać prawdziwe faktury, co naruszałoby przepisy RODO oraz uniemożliwiłoby automatyczną weryfikację poprawności, zaimplementowano własny generator dokumentów (`scripts/generator.py`).

Działanie skryptu, zobrazowane na rys. 3, opiera się na puli przygotowanych wcześniej danych słownikowych (ang. *mock data*), wczytywanych z plików `data/contractors.json` (listing 1) oraz `data/items.json`. Znajdują się w nich nazwy firm, adresy oraz definicje asortymentu (cena netto, stawka VAT). Główna funkcja `generate_payload()` za pomocą modułu `random` losuje pary sprzedawca-nabywca, generuje syntetyczne numery NIP, numery kont bankowych oraz dynamicznie oblicza daty sprzedaży, wystawienia i termin płatności wykorzystując moduł `datetime`. Na podstawie wylosowanej liczby produktów (od 1 do 8), skrypt matematycznie oblicza sumy częściowe i całkowite kwoty brutto/netto, eliminując ryzyko błędów księgowych w generowanych dokumentach.

Tak przygotowany obiekt danych (tzw. „słownik” w języku Python) przesyłany jest do silnika szablonów `Jinja2`, który kompiluje strukturę HTML do wybranego układu graficznego faktury (ang. *layout*). Następnie wykorzystywana jest biblioteka `pdfkit`, wywołująca proces `wkhtmltopdf` w celu bezstratnego wygenerowania pliku PDF. W ostatnim kroku biblioteka `pdf2image` (wspierana przez Popplera) renderuje dokument wektorowy do formatu rastrowego PNG o stałej rozdzielczości 300 DPI.

W celu zasymulowania różnych warunków rzeczywistych (dokumenty cyfrowe, skany z drukarki, zdjęcia z telefonu), skrypt został zaprogramowany w taki sposób, aby przez indeksację pętli kategoryzować wyjściowe obrazy do trzech odpowiednich folderów docelowych: `png_clean`, `png_scan` oraz `jpg_photo`. Równolegle z generowaniem obrazu, wylosowane dane referencyjne zapisywane są bezpośrednio do relacyjnej bazy danych SQLite za pomocą funkcji `save_ground_truth()`. Stanowi to absolutny wzorzec, czyli ground truth, do którego w fazie testowej porównywane będą wyniki odczytów z silników OCR.



Rys. 3. Schemat działania skryptu.

```

{
  "company_names": [
    "TechPol Sp. z o.o.",
    "Budex SA",
    "AgroMax",
    "MediCare Polska",
    "AutoSerwis Kowalski",
    "GreenEnergy",
    "SoftDev Solutions",
    "LogiTrans Polska",
    "FoodMarket Sp. z o.o.",
    "EduFuture Akademia",
    "PrintMax Drukarnia",
    "Hotel Luxor",
    "EventPro Organizacja Imprez",
    "Farmacia Zdrowie",
    "Księgowość Plus",
    "ArchDesign Studio",
    "Transport Express",
    "Sklep RTV AGD",
    "MotoParts",
    "EcoBuild"
  ],
  "addresses": [
    "ul. Piotrkowska 123, 90-001 Łódź",
    "ul. Długa 45, 00-950 Warszawa",
    "ul. Mickiewicza 12, 30-001 Kraków",
    "ul. Główna 7, 61-001 Poznań",
    "ul. Polna 88, 80-001 Gdańsk",
    "ul. Świdnicka 22, 50-001 Wrocław",
    "ul. Kościuszki 10, 20-001 Lublin",
    "ul. Słoneczna 5, 40-001 Katowice",
    "ul. Spacerowa 77, 70-001 Szczecin",
    "ul. Kwiatowa 15, 85-001 Bydgoszcz",
    "ul. Mazowiecka 33, 00-067 Warszawa",
    "ul. Wrocławska 99, 30-017 Kraków",
    "ul. Jana Pawła II 12, 25-001 Kielce",
    "ul. Leśna 8, 87-100 Toruń",
    "ul. Warszawska 55, 35-001 Rzeszów",
    "ul. Gdańska 101, 10-001 Olsztyn",
    "ul. Lubelska 44, 15-001 Białystok",
    "ul. Krótka 3, 45-001 Opole",
    "ul. Szeroka 18, 31-001 Kraków",
    "ul. Nowa 20, 60-001 Poznań"
  ]
}

```

Listing 1. Plik data/contractors.json.

2.4.2. Główne API systemu i zarządzanie żadaniami

Centralnym punktem wejściowym realizującym logikę przetwarzania dokumentów jest plik `app/main.py`. Zaimplementowano w nim pojedynczy punkt końcowy (rys. 4) (ang. *endpoint*) `/ocr`, obsługujący żądania typu POST.

The screenshot shows the FastAPI Swagger UI for the `/ocr` endpoint. The interface is titled 'default' and 'POST /ocr Ocr Endpoint'. It includes a 'Parameters' section with 'No parameters' and a 'Request body' section with a 'multipart/form-data' dropdown. The request body fields are: 'file' (required, string, with a file selection button), 'engine' (required, string, with a text input), 'layout' (required, string, with a text input and a 'Send empty value' checkbox), 'input_type' (required, string, with a text input and a 'Send empty value' checkbox), and 'test_mode' (boolean, dropdown menu set to 'false', with a 'Send empty value' checkbox). An 'Execute' button is at the bottom.

Rys. 4. Interfejs endpointu `/ocr` (FastAPI).

Z poziomu kodu, funkcja asynchroniczna `ocr_endpoint` przyjmuje strumień bajtów pliku graficznego (`UploadFile`) oraz niezbędne metadane w postaci pól formularza (`Form`): nazwę silnika (EasyOCR, Tesseract, Azure), identyfikator układu graficznego oraz typ wejścia (skan, zdjęcie, plik czysty). Aby uniknąć rozbudowy i trudnych w utrzymaniu instrukcji warunkowych przy wyborze silnika analizującego, zastosowano wzorec strategii w postaci słowników mapujących (`ENGINE_MAP_WORDS` oraz `ENGINE_MAP_TEXT`). Pozwala to na dynamiczne przekierowanie strumienia bajtów obrazu bezpośrednio do odpowiedniego adaptera na podstawie nazwy przekazanej w żądaniu. Wyjątkiem od tej reguły jest integracja z chmurą Azure, która z racji swojego zewnętrznego, sieciowego charakteru odpytywana jest w wydzielonym bloku operacyjnym.

Istotnym elementem implementacji punktu dostępowego jest wbudowany mechanizm profilowania wydajności. Moduł `time` przechwytuje znacznik czasowy bezpośrednio przez wywołaniem funkcji silnika OCR oraz tuż po jej zakończeniu. Wyliczona w ten sposób różnica, reprezentująca całkowity czas ekstrakcji danych w milisekundach, jest następnie przekazywana wraz z rozpoznanym tekstem i wygenerowanymi strukturami słów do funkcji `parse_invoice`. W ostatnim kroku, w przypadku, gdy zapytanie nie zostało wywołane w izolowanym trybie testowym (`test_mode`),

przetworzony komplet danych zlecany jest do zapisu asynchronicznego w bazie danych z wykorzystaniem funkcji `save_ocr_results`.

Realizacja warstwy persystencji (utrwalania danych) została wyodrębniona do pliku `app/db/crud_ocr.py`. Wspomniana funkcja odpowiada za dekompozycję zwróconego z parsera słownika na poszczególne obiekty mapowania ORM (zdefiniowane w pliku `models.py`, których schemat bazodanowy zaprezentowano wcześniej na rys. 2). W ramach jednej transakcji bazodanowej system tworzy powiązane ze sobą rekordy w Tab.ch: `OCRRun` przechowującej pełen log operacyjny z surowym tekstem przydatnym do debugowania, `OCRInvoice` z danymi bagłóvkowymi faktury oraz `OCRItem` dla wyekstrahowanego asortymentu. Istotnym krokiem jest tu również proces sanitizacji danych numerycznych. Zastosowana funkcja pomocnicza `_to_float()` ujednolica zdekodowane przez silniki OCR wartości finansowe zmieniając przecinki na kropki oraz usuwając białe znaki, a następnie bezpiecznie rzutuje je na format zmiennoprzecinkowy (*float*), zapobiegając błędom zapisu w bazie SQLite.

2.4.3. Moduł integracji silników OCR i post-processing

Każdy z badanych silników rozpoznawania tekstu charakteryzuje się odmiennym interfejsem programistycznym oraz formatem zwracanych danych. Aby zachować spójność wewnątrz systemu zgodnie z przyjętą architekturą, implementacja adapterów (np. `app/ocr/easyocr_adapter.py`) opiera się na bezplikowym dekodowaniu przestanych obrazów. Zamiast zapisywać pliki tymczasowe na dysku serwera, strumień danych wejściowych jest transformowany za pomocą biblioteki *numpy* do jednowymiarowej tablicy bajtów (ang. byte array), a następnie dekodowany przez bibliotekę *OpenCV* do postaci wielowymiarowej macierzy reprezentującej obraz w przestrzeni barw BGR, zrozumiałej dla algorytmów analitycznych. Takie podejście znacząco redukuje narzut wejścia/wyjścia (I/O). Wynikiem działania każdego adaptera jest ujednolicona lista słowników zawierająca wyodrębniony ciąg znaków oraz jego znormalizowane współrzędne graniczne (x,y,w,h)¹⁶.

Dla pozostałych narzędzi mechanizm ten wygląda nieco inaczej. Adapter Tesseracta (`app/ocr/pytesseract_adapter.py`) bazuje na bibliotece obrazów *PIL (Python IMaging Library)*. Przekazując obraz do procesu `tesseract.exe`, wykorzystuje argument `output_type=pytesseract.Output.DICT`, wymuszając zwrot struktury słownikowej, po której interuje odrzucając artefakty niemające rozpoznanego tekstu. Z kolei moduł Microsoft Azure (`app/ocr/azure_adapter.py`) realizuje trudną technicznie komunikację asynchroniczną z chmurą. Wykorzystując bibliotekę *requests*,

¹⁶ x, y to współrzędne punktu początkowego (najczęściej lewego górnego rogu) ramki otaczającej tekst (ang. *bounding box*), natomiast w (szerokość, ang. *width*) i h (wysokość, ang. *height*) określają wymiary tej ramki.

funkcja implementuje mechanizm `_safe_post`, który zarządza limitami zapytań (błąd `http 429`), dynamicznie wstrzymując wątek (`time.sleep`).

Po zleceniu analizy następnie proces cyklicznego odpytywania usługi (ang. *polling*) z wykorzystaniem nagłówka `Operation-Location`, aż do uzyskania pełnego dokumentu JSON z semantycznie zdekodowanymi polami.

Najbardziej złożonym etapem implementacji była warstwa post-processingu, na którą składają się dwa niezależne parsery (rys. 5):

1. Parser metadanych (`app/parsers/text_parser.py`): Jego zadaniem jest wyodrębnienie z surowego bloku tekstu kluczowych atrybutów faktury m.in. NIP, numer konta, sumy końcowe. Został on zaimplementowany przy pomocy modułu wyrażeń regularnych (`re`). Wykorzystano w nim flagi ignorujące wielkość liter (`IGNORECASE`) oraz skomplikowane asercje pozwalające wychwycić ciągi cyfr w konkretnym kontekście (np. dziesięciocyfrowy numer NIP znajdujący się bezpośrednio po słowach „Nabywca” lub „Sprzedawca”). Warto zaznaczyć, że zaprogramowana w całym module logika cechuje się dużą odpornością na błędy wynikające z jakości akwizycji obrazu – w szczególności na częste zjawisko gubienia pojedynczych liter przez silniki OCR. Problem ten został rozwiązany systemowo poprzez uwzględnienie w wyrażeniach regularnych wariantów słów z błędami zaobserwowanymi podczas testów. Jako reprezentatywny przykład takiego mechanizmu można wskazać funkcję `_extract_buyer_nip` (listing 2). Potrafi ona prawidłowo przyporządkować NIP, nawet jeśli słowo kluczowe „Nabywca” zostanie przez OCR zniekształcone do formy „Nabyca” czy „Nabwca” (wynikającej z gubienia liter "w" lub "y"). Podobne rozwiązanie zastosowano dla wielu innych słów kluczowych w dokumencie. Dodatkowo, przy wyszukiwaniu sum kontrolnych, algorytm priorytetyzuje kolumnowy układ kwot na dokumencie (netto, VAT, brutto obok siebie pionowo), a dopiero w razie niepowodzenia stosuje metodę dopasowania horyzontalnego.
2. Algorytm analizy przestrzennej (rys. 6) (`app/parsers/table_parser.py`): Moduł ten odpowiada za rekonstrukcję tabeli z pozycjami asortymentowymi, co wymaga analizy dwuwymiarowej geometrii dokumentu. W pierwszej fazie algorytm grupuje rozproszone słowa w wiersze, badając dystans między środkami wysokości poszczególnych elementów (ang. *row tolerance*, przyjęty w kodzie na poziomie 10 pikseli). Z powstałych wierszy skrypt identyfikuje nagłówki tabeli na podstawie zagęszczenia zdefiniowanych słów kluczowych (np. „ilość”, „j.m.”, „cena”, „netto”). Następnie, bazując na środkach ciężkości tych słów, wylicza granice kolumn i zastępująco dopasowuje do nich wykryte w niższych partiach wartości numeryczne. Wdrożono tu również reguły

heurystyczne radzące sobie z typowymi błędami OCR (np. automatycznie rozdzielanie zlepionych wartości „8godz” na osobną ilość i jednostkę miary).

Faktura VAT

Nr faktury: FV/8417

Data wystawienia: 2026-01-28 **Data sprzedaży:** 2026-01-26 **Waluta:** PLN

SPRZEDAWCA

ArchDesign Studio
ul. Szeroka 18, 31-001 Kraków
NIP: 7107959125
REGON: 704635153
Konto: PL58814323400384257000897100

NARYWCA

EventPro Organizacja Imprez
ul. Słoneczna 5, 40-001 Katowice
NIP: 7139115739
Numer zamówienia: ORD-883
Termin płatności: 2026-02-05

Lp.	Pozycja	Ilość	J.m.	Cena netto	VAT	Netto	Brutto
1	Papier A4 – ryza	9	ryza	18	23	162	199.26
2	Usługa serwisowa IT	9	godz.	150	23	1350	1660.5
3	Naprawa sprzętu AGD	1	usługa	300	8	300	324.0

Suma netto 1812

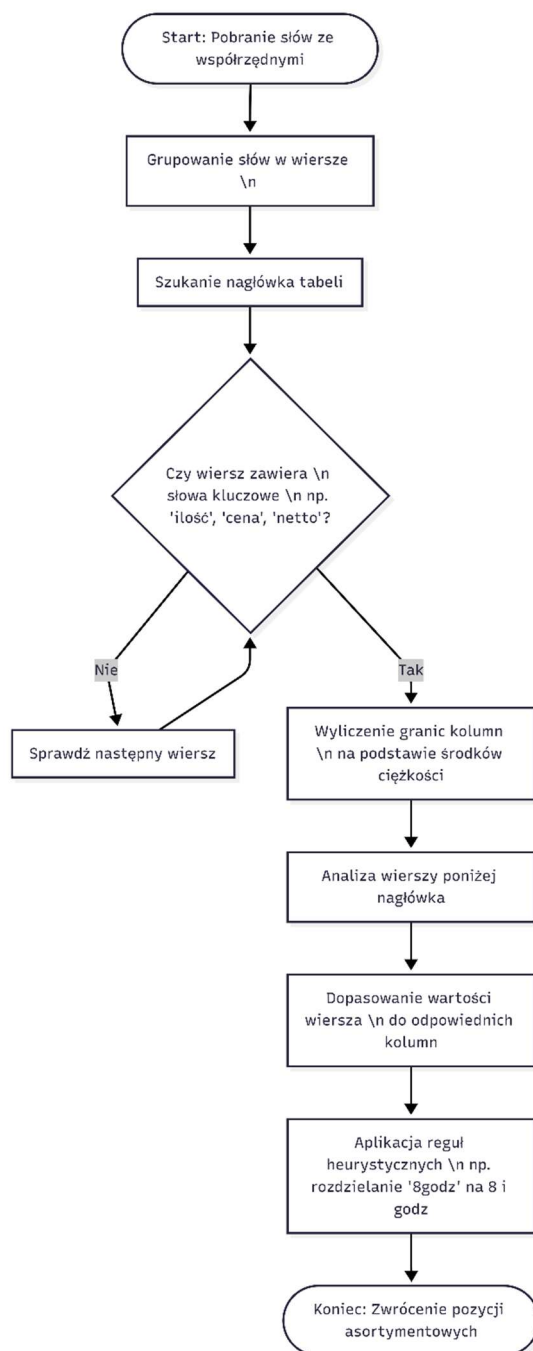
Suma VAT 371.76

Suma brutto 2183.76

Sposób zapłaty: PayPal

Uwagi: Dziękujemy za współpracę

Rys. 5. Wizualna reprezentacja działania parserów.



Rys. 6. Blokowa reprezentacja algorytmu analizy przestrzennej.

```
def _extract_buyer_nip(raw_text: str):
    m = re.search(
        r"(NABYWCA|Nabywca|NABYCA|Nabyca)(.*?)(NIP[:\s]*([0-9]{10}))",
        raw_text,
        flags=re.IGNORECASE | re.DOTALL,
    )
    if m:
        return m.group(4)

    all_nips = re.findall(r"NIP[:\s]*([0-9]{10})", raw_text, re.IGNORECASE)
    if len(all_nips) >= 2:
        return all_nips[1]

    return None
```

Listing 2. Funkcja `_extract_buyer_nip`

2.4.4. Automatyzacja testów i analiza wyników

Zanim uruchomiona główny skrypt ewaluacyjny, na potrzeby weryfikacji manualnej oraz efektywnego debugowania parserów stworzono narzędzie wiersza poleceń `scripts/client_submit.py`. Skrypt ten przyjmuje parametry z wykorzystaniem wbudowanej biblioteki `argparse`, dynamicznie rozpoznaje format pliku z katalogu testowego i wysyła izolowane zapytanie do serwera API. Umożliwiło to bezpieczne badanie pojedynczych wygenerowanych szablonów poprzez zrzut szczegółowej struktury komunikacyjnej JSON do plików w trybie deweloperskim.

Ostatnim etapem prac implementacji było wytworzenie narzędzi umożliwiających przeprowadzenie masowych testów dla setek przygotowanych próbek, bez konieczności ingerencji użytkownika. Zrealizowano to za pomocą skryptu wykonywalnego `scripts/batch_test.py`. Skrypt ten przyjmuje jako argument ścieżkę do głównego katalogu z wynikami generatora, a następnie w sposób zagnieżdżony iteruje po folderach podziału (`png_clean`, `jpg_photo`, `png_scan`). Dla każdego napotkanego pliku graficznego algorytm odczytuje nazwę użytego szablonu i nawiązuje asynchroniczne połączenie sieciowe z lokalnym serwerem API przy pomocy biblioteki `requests`. Każdy obraz zostaje przetworzony trzykrotnie (z wymuszeniem innego silnika parametryzowanego w ciele żądania POST). Surowe logi z odpowiedziami serwera są dodatkowo serializowane do pliku `all_results.json`, zapewniając kopię zapasową w razie awarii bazy danych lub nieoczekiwanego przerwania testów.

Weryfikacja jakości zaimplementowanych rozwiązań została zaprogramowana w autonomicznym skrypcie `scripts/benchmark.py`. Nawiązuje on połączenie z bazą `invoices.db` i krzyżowo odpytuje tabele `ground_truth_invoices` oraz `ocr_invoices`. Wykorzystując struktury słownikowe języka Python oraz bezpośrednie zapytania SQL za pomocą modułu `sqlite3` (w celu ominięcia narzutów wydajnościowych warstwy ORM), skrypt pole po polu sprawdza poprawność

odczytywanych znaków w relacji do oryginału. Dla wartości tekstowych system analizuje skuteczność odczytu, podczas gdy dla kwot finansowych i Tab.rycznych zestawień produktów algorytm posiada złożoną logikę matematyczną. Wykonuje on zliczenia poprawnych podporządkowań (True Positives), wychwytuje brakujące pola (False Negatives) oraz nadmiarowo przypisane szumy (False Positives). Ostatecznym wynikiem jego działania jest ustrukturyzowany raport, agregujący czas procesowania, wskaźniki błędów bezwzględnych kwot (MAE) oraz przeliczone z wzorów wskaźniki F1-score dla każdego silnika z podziałem na stopień zniekształcenia pliku wejściowego.

2.5. Instrukcja uruchomienia i obsługi programu

Z uwagi na integrację języka wysokiego poziomu Python z zewnętrznymi plikami wykonywalnymi (binariami systemowymi) oraz usługami chmurowymi, poprawne uruchomienie zaimplementowanego systemu wymaga przeprowadzenia ścisłej konfiguracji środowiska uruchomieniowego. System został zoptymalizowany pod kątem pracy w środowisku Windows. Poniższa instrukcja szczegółowo opisuje proces wdrożenia z wykorzystaniem interfejsu wiersza poleceń (CLI).

1. Pobranie kodu źródłowego (system kontroli wersji Git)

Całość kodu źródłowego projektu jest zarządzana za pomocą systemu kontroli wersji Git i hostowana na platformie GitHub. Proces wdrożenia należy rozpocząć od sklonowania repozytorium na dysk lokalny komputera oraz przejścia do tworzonego katalogu roboczego. W oknie terminala należy wykonać następujące polecenia:

```
git clone https://github.com/Alex-U02/Inz.git  
cd Inz
```

2. Instalacja zależności systemowych

Do poprawnego działania modułów przetwarzających dokument PDF oraz warstwy OCR konieczna jest manualna instalacja oprogramowania firm trzecich w systemie operacyjnym Windows:

- Tesseract-OCR:

Należy pobrać oficjalny instalator binarnego silnika Tesseract. Podczas instalacji bezwzględnie wymaga się dołączenia pakietu wsparcia dla języka polskiego (`pol.traineddata`). Plik wykonywalny musi zostać zainstalowany w katalogu zgodnym z konfiguracją projektu:

```
C:\Program Files\Tesseract-OCR\tesseract.exe.
```

- Wkhtmltopdf:

Oprogramowanie niezbędne do bezstratnego renderowania szablonów HTML do formatu PDF. Należy je zainstalować w domyślnej ścieżce systemowej:

```
C:\Program Files\wkhtmltopdf\bin\wkhtmltopdf.exe.
```

- **Poppler:**
Biblioteka służąca do rasteryzacji dokumentów wektorowych. Skompilowane archiwum (w wersji zgodnej z 24.02.0) należy rozpakować do katalogu:
`C:\Program Files\poppler-24.02.0\.`
Krytycznym etapem jest ręczne dodanie podkatalogu `\Library\bin` do globalnej zmiennej środowiskowej systemu Windows (zmienna `PATH`), co umożliwi interpreterowi Pythona poprawne zlokalizowanie narzędzi renderujących.

3. Konfiguracja wirtualnego środowiska Python

W celu uniknięcia konfliktów pomiędzy bibliotekami systemowymi, aplikację należy uruchomić w izolowanym środowisku wirtualnym (`venv`). W głównym katalogu (`Inz`), korzystając z terminala, należy wykonać sekwencję poleceń:

- Instalacja środowiska:
`python -m venv .venv`
- Aktywacja środowiska dla powłoki systemu Windows:
`.\.venv\Scripts\activate`
- Instalacja pakietów programistycznych:
`pip install -r requirements.txt`

4. Uruchomienie serwera aplikacji

Po przygotowaniu zależności, proces badawczy wymaga uruchomienia głównego serwera API, który będzie zarządzał potokiem przetwarzania danych. W aktywnym środowisku wirtualnym należy wywołać serwer ASGI za pomocą pakietu Uvicorn, który został uprzednio zainstalowany wraz z resztą zależności z pliku `requirements.txt`:

```
uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Serwer zaalokuje port 8000 i w sposób automatyczny zainicjalizuje plikową bazę danych SQLite. Pod ścieżką sieciową <http://127.0.0.1:8000/docs> udostępniona zostanie specyfikacja API w standardzie Swagger UI.

5. Przeprowadzenie procesu badawczego i ewaluacji

Zautomatyzowane scenariusze testowe wykonuje się w osobnym oknie terminala pamiętając o uprzedniej aktywacji środowiska wirtualnego. Wszystkie narzędzia analityczne wywołane są z poziomu głównego katalogu projektu jako moduł języka Python. Gwarantuje to poprawne rozwiązanie ścieżek importu pomiędzy niezależnymi pakietami aplikacji.

- Generowanie bazy referencyjnej (ground truth):
`python -m scripts.generator`

Polecenie tworzy pulę syntetycznych faktur i zapisuje je w zagnieżdżonych strukturach katalogu docelowego `out/`.

- Manualna akwizycja danych fizycznych:

Ponieważ skrypt generatora tworzy wyłącznie idealne pliki cyfrowe, na tym etapie wymagana jest interwencja operatora. Aby zbadać wpływ rzeczywistych zakłóceń, dokumenty wygenerowane do folderów `png_scan` oraz `jpg_photo` należy wydrukować, a następnie fizycznie zeskanować oraz sfotografować. Uzyskane w ten sposób obrazy należy umieścić z powrotem w odpowiednich katalogach podmieniając pliki wygenerowane automatycznie. Kluczowe jest rygorystyczne zachowanie pierwotnych nazw plików, ponieważ na ich podstawie skrypt ewaluacyjny złączy wyniki ekstrakcji z danymi wzorcowymi w bazie SQLite.

- Procesowanie wsadowe (*batch test*):

```
python -m scripts.batch_test --root out --clear
```

Skrypt iteruje po przygotowanych w poprzednim kroku dokumentach w folderze głównym (`out`) i przesyła je do lokalnego serwera API. Flaga `clear` jest niezbędna do usunięcia historycznych danych z bazy `invoices.db` przed rozpoczęciem nowej próby badawczej. Analiza zostanie wykonana w pętli dla każdego silnika, a jej zrównoleglone wyniki zostaną zarchiwizowane w bazie SQLite oraz w awaryjnym pliku `all_results.json`.

- Generowanie raportu (*benchmark*):

```
python -m scripts.benchmark
```

Zwieńczeniem procesu jest ewaluacja matematyczna, weryfikująca odczyty API względem bazy referencyjnej (*ground truth*). Skrypt oblicza zaimplementowane metryki statystyczne i eksportuje je do pliku tekstowego `benchmark_report.txt` (rys. 7) dostarczając ostateczny materiał do dyskusji nad skutecznością badanych systemów w następnym rozdziale pracy.


```

--- DOKŁADNOŚĆ PÓŁ FAKTURY ---

easyocr/clean: dokładność=0.887, correct=674, wrong=86, missing=7
easyocr/photo: dokładność=0.558, correct=424, wrong=336, missing=213
easyocr/scan: dokładność=0.730, correct=555, wrong=205, missing=98
pytesseract/clean: dokładność=0.921, correct=700, wrong=60, missing=41
pytesseract/photo: dokładność=0.796, correct=605, wrong=155, missing=111
pytesseract/scan: dokładność=0.795, correct=604, wrong=156, missing=94
azure/clean: dokładność=0.634, correct=482, wrong=278, missing=278
azure/photo: dokładność=0.626, correct=476, wrong=284, missing=281
azure/scan: dokładność=0.629, correct=478, wrong=282, missing=282

--- DOKŁADNOŚĆ POZYCJI TOWAROWYCH ---

easyocr/clean: precyzja=0.705, czułość=0.705, F1=0.705 (tp=117, fp=49, fn=49)
easyocr/photo: precyzja=0.350, czułość=0.291, F1=0.318 (tp=62, fp=115, fn=151)

```

Rys. 7. Fragment pliku *benchmark_report.txt*.

3. Badania skuteczności i wydajności systemów OCR

Rozdział ten poświęcony jest prezentacji, analizie oraz dyskusji nad wynikami przeprowadzonych eksperymentów badawczych. Wykorzystując autorski mikroserwis i zautomatyzowane skrypty testowe opisane w poprzednim rozdziale, zbadano skuteczność trzech silników OCR: Tesseract, EasyOCR oraz Microsoft Azure AI.

3.1. Scenariusze i środowisko testowe

Wiarygodność przeprowadzonych pomiarów wydajnościowych uzależniona jest od środowiska uruchomieniowego. Testy lokalnych silników (Tesseract, EasyOCR) przeprowadzono na jednostce roboczej wyposażonej w procesor Intel Core i5-13600KF oraz w 32 GB pamięcią RAM, przy czym EasyOCR korzystał również z akceleracji na karcie graficznej Nvidia GeForce RTX 3060 Ti. W celach badawczych, w skrypcie *easyocr_adapter.py* w jednym z wariantów testowych jawnie wymuszono wykonywanie obliczeń macierzowych głębokich sieci neuronowych na procesorze głównym (flaga `gpu=False`), aby zrównoważyć porównanie z klasycznym silnikiem Tesseract. W drugim wariancie pozostawiono akcelerację GPU włączoną, umożliwiając EasyOCR

Zgodnie z przyjętą metodologią, skrypt `scripts/generator.py` posłużył do tworzenia puli dokumentów na podstawie 8 różnych układów graficznych (tzw. „layoutów”) (rys. 8). Dokumenty te zostały rozdzielone na trzy zbiory wejściowe:

- Cyfrowe pliki oryginalne (*clean*) – wygenerowane bezpośrednio do bezstratnego formatu graficznego.
- Dokumenty skanowane (*scan*) – w celu uzyskanie tego zbioru, oryginalne faktury zostały najpierw fizycznie wydrukowane, a następnie zdigitalizowane (zeskanowane w rozdzielczości 300 DPI) za pomocą urządzenia wielofunkcyjnego *Brother DCP-T510W*. Pozwoliło to na wprowadzenie do badanej próby naturalnych zniekształceń wynikających z fizycznej struktury papieru oraz mikroskopijnymi niedoskonałościami optyki skanera.
- Zdjęcie mobilne (*photo*) – wydrukowane dokumenty za pomocą urządzenia wielofunkcyjnego *Brother DCP-T510W*, po czym sfotografowane przy pomocy wbudowanego aparatu w smartfonie *Samsung Galaxy S23*, wprowadzające naturalne zakłócenia (cienie, zagięcia papieru, refleksy świetlne).

Zautomatyzowany skrypt `batch_test.py` wykonał łącznie 360 asynchronicznych iteracji badawczych, po 40 prób na każdy z trzech silników i trzech typów zniekształceń. Zgromadzone dane, wyciągnięte za pomocą `benchmark.py`, stanowią podstawę poniższych ewaluacji.

[illegible][illegible]

32

3.2. Wyniki testów wydajnościowych

Pierwszym z badanych aspektów była wydajność czasowa, mierzona od momentu przyjęcia pliku graficznego do interfejsu API do momentu wygenerowania sparsowanego ustrukturyzowania dokumentu JSON. Zmienna ta była zagregowana w bazie danych pod postacią atrybutu `duration_ms`. Wyniki 40 prób dla kolejnych silników zaprezentowano w tab. 1.

	EasyOCR (gpu=False)			EasyOCR (gpu=True)			PyTesseract			Azure		
	clean	photo	scan	clean	photo	scan	clean	photo	scan	clean	photo	scan
średnia [ms]	14761,1	15235,8	13031,1	2791,9	3003,5	2282,4	1683,6	1826,5	1596,9	5406,7	7119,7	5843,8
min [ms]	13075	13431	11893	2195	2200	1808	1443	1252	1072	3407	3716	3613
max [ms]	17184	17423	15118	3611	3693	2872	2051	2553	2105	11695	13255	12360

Tab. 1. Wyniki `duration_ms` dla kolejnych silników.

Z uzyskanych raportów bezdyskusyjnym zwycięzcą pod kątem szybkości procesowania okazał się klasyczny silnik Tesseract OCR. Osiągnął on rewelacyjny średni czas analizy wynoszący zaledwie 1683.6 ms dla plików cyfrowych (*clean*), 1596.9 ms dla skanów oraz 1826.5 ms dla zdjęć mobilnych. Skrajne odchylenia (maksymalne 2553 ms) nadal mieściły się w granicach akceptowalnych dla systemów czasu rzeczywistego. Wynik ten dowodził jego wysokiej optymalizacji pod kątem operowania na lokalnych zasobach systemowych.

Rozwiązanie chmurowe Microsoft Azure charakteryzowało się umiarkowanym, aczkolwiek akceptowalnym czasem procesowania. Średnie wartości wahały się od 5406.7 ms (*clean*) do 7119.7 ms (*photo*). Warto jednak zauważyć, że narzut ten w głównej mierze nie wynika z powolności samych algorytmów sztucznej inteligencji, lecz jest bezpośrednim skutkiem latencji sieciowej oraz zastosowanego w kodzie adaptera mechanizmu *polling* - cyklicznego odpytywania zewnętrznego serwera API o status zleconego zadania asynchronicznego [24].

Najgorzej w zestawieniu wydajnościowym wypadł pakiet EasyOCR, który działał jedynie na procesorze centralnym (`gpu=False`). Ze względu na wykorzystywanie bardzo ciężkich modeli uczenia głębokiego (CRAFT do detekcji oraz CRNN do rozpoznawania znaków) dekompilowanych na architekturze procesora centralnego (CPU), średni czas przetwarzania pojedynczej faktury wynosił od 13031.1 ms (*scan*) aż do 15235.8 ms (*photo*). Narzut rzędu 15 sekund na pojedynczy dokument może stanowić barierę zaporową w systemach produkcyjnych przetwarzających masowe potoki danych.

Sytuacja ma się inaczej, gdy EasyOCR wykorzystywał zarówno procesor centralny (CPU) oraz procesor graficzny (GPU, flaga `gpu=True`). Rozwiązanie to osiągnęło wówczas dużo lepsze wyniki uzyskując średni czas analizy $2791,9\text{ ms}$ dla plików *clean*, $3003,5\text{ ms}$

dla zdjęć oraz 2282,4 ms dla skanów. Porównując wyniki z wyłączoną i włączoną flagą `gpu`, zauważyć można, iż dzięki większej mocy obliczeniowej wynikającej z dodatkowej jednostki przetwarzającej, pakiet EasyOCR stał się około 5 razy wydajniejszy czasowo, co dowodzi, iż może on stanowić dobrego konkurenta dla pozostałych rozwiązań badanych w niniejszej pracy, zakładając, że użytkownik dysponuje jednostką GPU.

3.3. Wyniki testów jakościowych

Ocenę jakości ekstrakcji zrealizowano na podstawie zautomatyzowanego skryptu ewaluacyjnego weryfikowanego w sposób krzyżowy zgodność odczytywania znaków z wygenerowaną bazą referencyjną (ground truth). Aby podjąć próbę wyjaśnienia zidentyfikowanych odchyleń, surowe wyniki statystyczne skorelowano z logami przestrzenno-tekstowymi zapisanymi w pliku `all_results.json` oraz przeanalizowano strukturę kodu źródłowego zaimplementowanych szablonów.

	EasyOCR			PyTesseract			Azure		
	clean	photo	scan	clean	photo	scan	clean	photo	scan
dokładność	0,887	0,558	0,73	0,921	0,796	0,795	0,634	0,626	0,629
correct	674	424	555	700	605	604	482	476	478
wrong	86	336	205	60	155	156	278	284	282
missing	7	213	98	41	111	94	278	281	282

Tab. 2. Dokładność pól faktury.

Rozpoznawanie atrybutów statycznych (metadane)

Badanie wskaźnika dokładności (*accuracy*) (tab. 2) dla statycznych nagłówków faktury obnażyło ogromną wrażliwość algorytmów lokalnych na zniekształcenia geometryczne. Silnik Tesseract na idealnych, wektorowych plikach cyfrowych (clean) osiągnął imponującą precyzję rzędu 92.1% (700 poprawnych dopasowań). W przypadku pakietu EasyOCR na tym samym zbiorze dokładność wynosiła 88.7%. Weryfikacja nastąpiła na wymagającym zbiorze *photo*. Dokładność Tesseracta spadła tam do 79.6%, natomiast EasyOCR odnotował drastyczny regres do zaledwie 55.8% (336 wartości błędnych i 213 całkowicie pominiętych pól). Wynik ten jest bezpośrednim efektem braku wbudowanych w te biblioteki mechanizmów wyrównywania obrazu (*de-skewing*). Nawet delikatny obrót kartki na zdjęciu ze smartfona zaburzał heurystykę autorskiego parsera.

Z kolei wyniki usługi chmurowej Microsoft Azure oscylowały na stabilnym poziomie 62.6% - 63.4%. Wartość ta jest efektem przyjętej rygorystycznej metodologii badawczej i architektury samego modelu dostawcy. Prekonfigurowany model faktur Microsoftu z założenia nie obsługuje ekstrakcji 4 z 19 badanych atrybutów, przykładowo takich jak daty sprzedaży czy numeru REGON, zwracając dla nich wartości puste. Skrypt oceniał

chmurę względem pełnej bazy wzorcowej, co automatycznie obniżało jej maksymalną teoretyczną dokładność do około 78%. W ujęciu relatywnym wynik chmury jest zatem wysoce satysfakcjonujący, ponieważ jako jedyny nie uległ degradacji pod wpływem zniekształceń fotograficznych.

Zależność rekonstrukcji tabeli od użytej typografii i układu HTML

Kluczowym odkryciem było skorelowanie skuteczności parsowania tabel (F1-Score) i błędów kwotowych (MAE) z kodem konkretnych szablonów (tab. 3).

	EasyOCR			PyTesseract			Azure		
	clean	photo	scan	clean	photo	scan	clean	photo	scan
precyzja	0,705	0,35	0,572	0,664	0,7	0,655	0,741	0,742	0,783
czułość	0,705	0,291	0,572	0,616	0,636	0,6	0,741	0,742	0,783
F1	0,705	0,318	0,572	0,639	0,667	0,626	0,741	0,742	0,783
tp	117	62	103	101	14	108	123	158	141
fp	49	115	77	51	6	57	43	55	39
fn	49	151	77	63	8	72	43	55	39
MAE	0	323,6	3,403	427,7	113,3	67,07	257,8	311	308,7
MRE	0	5,268	0,017	0,384	0,214	2,246	0,064	0,064	0,066

Tab. 3. Dokładność pozycji towarowych i wartości liczbowych.

Wyizolowanie zmiennych graficznych pozwala wysnuć istotne hipotezy na temat podatności algorytmów na konkretne zabiegi wizualne:

1. Przypuszczalny wpływ czcionek szeryfowych (layout 2):
Zestawiając ze sobą standardowy layout 1 (font „Arial”) oraz layout 2 (font „Times New Roman”) zauważono drastyczną różnicę. Dla layoutu 1 błąd odczytu kwot w silniku Tesseract wyniósł 0.0 PLN, podczas gdy w layoutcie 2, przy zachowaniu tożsamej struktury tabeli HTML, błąd ten osiągnął wartość 1658.4 PLN na idealnie czystych plikach. Pozwala to z dużym prawdopodobieństwem założyć, że ozdobniki na końcach znaków (tzw. „szeryfy”) zlewają się silnikowi optycznemu z interpunkcją, co skutkuje przesuwaniem miejsc dziesiętnych przez algorytm przetwarzający.
2. Załamanie analizy na fontach monospacyjnych (layout 7):
Zastosowanie fontu „Courier New” wydaje się krytycznie obniżać skuteczność EasyOCR. Parametr F1-Score spadł tam na skanach do poziomu zaledwie 4.0%. Inspekcja węzłów tekstowych w pliku `all_results.json` pozwala przypuszczać, że przez nienaturalnie szerokie odstępy w wyrazach, EasyOCR

generował oddzielne obiekty przestrzenne (tzw. „bounding boxes”) dla niemal każdej pojedynczej litery, co znacznie utrudniało lub wręcz uniemożliwiało warstwie backendowej złożenie ich z powrotem w poprawne nazwy asortymentu.

3. Podatność analizy osi Y na zniekształcenia fotograficzne:

Krzyżowa analiza ujawniła, że dla 7 z 8 badanych układów, miara F1-Score w Tesseractcie na zbiorze *photo* wyniosła dokładnie 0.0 (rys. 9). Wgląd w macierz wyników JSON ([*"words"*]) pozwala sformułować logiczną tezę, iż z powodu minimalnego obrotu kartki na zdjęciu, słowo na początku linii posiadało często inną wartość rzędnej (np. $y = 1200$) niż kwota przypisana na końcu tej samej linii (np. $y = 1218$), co obrazuje rys. 10. Różnica ta, przekraczająca próg błędu zdefiniowany w parametrach funkcji *table_parser.py*, prawdopodobnie całkowicie niszczyła logikę grupowania wyrazów w jeden wiersz. W efekcie miara F1-Score w Tesseractcie spadła do poziomu dokładnie 0.0 (dla 7 z 8 układów), a w pakiecie EasyOCR odnotowała drastyczny regres, pikując w wielu układach do wartości rzędu 12-19%. Anomalia ta nie dotyczyła usługi Microsoft Azure, ponieważ rozwiązanie chmurowe samodzielnie analizuje strukturę przestrzenną i zwraca gotowe zestawienie wierszy [25], całkowicie omijając lokalny kod parsujący na serwerze aplikacji.

```
### Silnik: pytesseract / Typ: photo

Layout layout1: pola=0.926, F1=0.718, MAE=72.6 (fields_total=95)
Layout layout2: pola=0.726, F1=0.000, MAE=0.0 (fields_total=95)
Layout layout3: pola=0.779, F1=0.000, MAE=0.0 (fields_total=95)
Layout layout4: pola=0.789, F1=0.000, MAE=1660.5 (fields_total=95)
Layout layout5: pola=0.905, F1=0.000, MAE=0.0 (fields_total=95)
Layout layout6: pola=0.779, F1=0.000, MAE=0.0 (fields_total=95)
Layout layout7: pola=0.747, F1=0.000, MAE=0.0 (fields_total=95)
Layout layout8: pola=0.716, F1=0.000, MAE=0.0 (fields_total=95)
```

Rys. 9. Wyniki F1-score dla silnika PyTesseract dla obrazów typu *photo*.

4. Załamanie parsera liniowego przez asymetryczny CSS (layout 4 i 8):
W layoutcie 4 oraz 8 tabela podsumowująca została zagnieżdżona w osobnym tagu `<div>` i przesunięta asymetrycznie atrybutem `width: 55%; margin-left: auto;`. Wiele wskazuje na to, że właśnie ten zabieg złamał lokalną heurystykę liniową na czystych wektorach – Tesseract odnotował dla nich odczyty rzędu

zaledwie 29.4% i 21.4%. Silnik Azure, operujący na wielowymiarowej reprezentacji grafu całego obrazu, powiązał sekcje znacznie lepiej, notując w tych układach F1 na poziomie odpowiednio 78.9% i 63.2%.

Lp.	Pozycja	Ilość	J.m.	Cena netto	VAT	Netto	Brutto
1	Bilet lotniczy krajowy	3	szt.	350	8	1050	1134.0


```

{
  "text": "Lp.",
  "x": 251.0,
  "y": 1265.0,
  "w": 49.0,
  "h": 41.0,
  "confidence": 0.96
},

```

```

{
  "text": "Brutto",
  "x": 2150.0,
  "y": 1285.0,
  "w": 102.0,
  "h": 28.0,
  "confidence": 0.96
},

```

Rys. 10. Porównanie wartości y dla dwóch rozpoznanych słów z jednego wiersza.

5. Błąd numeryczny (MAE) a potencjalny dysonans semantyczny chmury: Mimo doskonałej orientacji przestrzennej, chmura Azure notowała powtarzalne uśrednione błędy (MAE) rzędu 257 - 310 PLN. Zrzuty `all_results.json` silnie sugerują jednak, że błąd ten nie leżał w samej jakości wizyjnego rozpoznawania cyfr. Międzynarodowy model Azure prawdopodobnie zakłada jedną, generyczną kwotę towaru. Zderzenie z polskim, czterokolumnowym formatem (cena netto, VAT, wartość netto, brutto) powodowało widoczną w logach anomalię - Azure często poprawnie odczytywał sumy dla całej faktury, lecz wewnątrz konkretnych pozycji towarowych ostateczną kwotę brutto podkładał pod uniwersalny klucz *Amount*, omijając dedykowane pole podatkowe *AmountWithTax* (listing 3, listing 4, rys. 11). Ponieważ skrypt `benchmark.py` rygorystycznie poszukiwał wartości netto pod kluczem *Amount*, traktował to jako błąd wartości, co sztucznie mogło zawyżyć statystykę MAE. Dla porównania, pakiet EasyOCR zdekodował kwoty z wektorowych plików bezbłędnie ($MAE = 0.0\text{ PLN}$), lecz na niedoświetlonych zdjęciach mobilnych fizyczne błędy rozpoznawania cyfr wywindowały jego odchylenie do poziomu 323.6 PLN.

```

items.append({
    "name": val(obj.get("Description")),
    "qty": val(obj.get("Quantity")),
    "unit": val(obj.get("Unit")),
    "price_net": val(obj.get("UnitPrice")),
    "vat": val(obj.get("TaxRate")),
    "net": val(obj.get("Amount")),
    "gross": val(obj.get("AmountWithTax")),
})

```

Listing 3. Fragment kodu `azure_adapter.py`.

Lp.	Pozycja	Ilość	J.m.	Cena netto	VAT	Netto	Brutto
1	Papier A4 – ryza	9	ryza	18	23	162	199.26

Rys. 11. Fragment rozpatrywanej faktury.

```
{
    "name": "Papier A4 - ryza",
    "qty": 9,
    "unit": "ryza",
    "price_net": "18",
    "vat": "23",
    "net": "199.26",
    "gross": null
},
```

Listing 4. Fragment pliku *all_results.json* z błędnie przypisanymi wartościami, silnik Azure.

3.4. Dyskusja wyników

Zgromadzone i skorelowane dane ewaluacyjne stwarzają pole do wieloaspektowej dyskusji nad użytecznością badanych technologii w rzeczywistych systemach. Wyniki eksperymentów obnażają fundamentalne różnice architektoniczne pomiędzy klasycznym podejściem heurystycznym opartym na współrzędnych wizyjnych, analityką bazującą na lokalnych sieciach neuronowych oraz chmurowymi rozwiązaniami dostarczającymi gotowe struktury danych.

Lokalne silniki klasyczne reprezentowane przez Tesseract, charakteryzują się bezkonkurencyjnym narzutem czasowym. Średni czas przetwarzania rzędu 1.6 sekundy na dokument stanowi optymalny wybór dla architektur operujących na tysiącach wygenerowanych elektronicznie i znormalizowanych plików cyfrowych (tzw. „born-digital”). Niestety, wyniki z podziałem na layouty i typy wejścia ujawniają podstawowe słabości całego potoku przetwarzania przestrzennego. Zjawisko całkowitego rozpadu tabel (wskaźnik $F1 = 0.0$) dla zbioru *photo* udowadnia, że budowa autorskich parserów opartych na tolerancji współrzędnych osi Y (jak wdrożony *table_parser.py*) jest skrajnie podatna na błędy fizyczne. Bez implementacji zaawansowanej warstwy wyrównującej obraz przed analizą (*de-skewing*), nawet ułamkowy obrót obiektywu niszczy logikę grupowania wierszy. Co więcej, załamanie skuteczności parsera na idealnych plikach cyfrowych w układach asymetrycznych (layout 4 i 8) oraz błędy odczytu przy fontach szeryfowych (layout 2) prowadzą do wniosku biznesowego, że lokalne systemy OCR wymagają od firm rygorystycznej standaryzacji szablonów dokumentów na prosty, symetryczny CSS z fontami bezszeryfowymi typu „Arial”.

Technologie w całości oparte na głębokich sieciach neuronowych (EasyOCR) rozwiązują część problemów związanych z odczytem numerycznym – model ten osiągnął bezbłędny wskaźnik $MAE = 0.0 PLN$ na plikach wektorowych. Jednakże wyniki z logów ukazują jego dwie istotne wady. Po pierwsze, podobnie jak Tesseract, jest on wrażliwy na typografię – rozpad wskaźnika F1 do 4% na fontach monospacyjnych dowodzi problemów algorytmu z poprawnym domykaniem ramek (tzw. „bounding boxes”) wokół znaków o nienaturalnych odstępach. Po drugie, narzędzie to natrafia na krytyczną barierę wydajnościową (ang. *hardware bottleneck*), jeśli nie ma dostępu do procesora graficznego jako dodatkowej jednostki przetwarzającej. Czas przetwarzania pojedynczego dokumentu na poziomie 13 do 15 sekund przy wykorzystaniu procesora centralnego (CPU) deklasuje to rozwiązanie w kontekście systemów czasu rzeczywistego. Skalowanie takiej architektury wymagałoby kosztownej inwestycji w serwery wyposażone w akceleratory graficzne (GPU), wówczas wydajność czasowa tego rozwiązania znacznie wzrasta i oscyluje w okolicy 2-3 s na dokument.

Rozwiązania chmurowe klasy Enterprise (Azure AI) prezentują zupełnie inny paradygmat architektoniczny. Utrzymanie stabilnego wskaźnika F1-Score (74% - 78%) niezależnie od faktu, czy wejściem był idealny PDF z asymetrycznym układem CSS, czy obrócone zdjęcie, wynika z faktu, że usługa ta nie wymaga lokalnych autorskich parserów. Jak dowodzi kod adaptera, Azure samodzielnie realizuje analizę wielowymiarową i zwraca do aplikacji gotową tablicę obiektów. Z drugiej strony, dyskusja nad zidentyfikowanymi błędami MAE (257 - 310 PLN) oraz zatrzymaniem ogólnej skuteczności na poziomie ok. 63% odkrywa ograniczenia gotowych modeli typu „pre-built”. Usługa Microsoftu, trenowana na globalnych, uniwersalnych wzorcach, cierpi na analityczne uprzedzenia w zderzeniu z polskim prawem podatkowym. Brak obsługi numeru REGON czy daty sprzedaży oraz notoryczne wstawianie wartości brutto pod generyczny klucz *Amount* udowadniają, że najdroższe i najbardziej zaawansowane narzędzia nie są w pełni autonomiczne. Wymagają one ze strony architekta oprogramowania ścisłego mapowania semantycznego oraz wdrożenia logiki korygującej po stronie aplikacji docelowej.

Podsumowując powyższe obserwacje, proces ekstrakcji danych z faktur jest zawsze kompromisem. Systemy lokalne oferują szybkość i brak kosztów subskrypcyjnych, lecz przerzucają na programistę ogromny koszt utrzymania i standaryzacji parserów geometrycznych. Systemy chmurowe gwarantują najwyższą jakość analizy układu, jednak zmuszają do ponoszenia stałych kosztów opóźnień sieciowych (ang. *API polling*) oraz walki z generycznymi ograniczeniami globalnych modeli sztucznej inteligencji.

Wnioski i podsumowanie

Niniejsza praca inżynierska miała na celu zaprojektowanie oraz przetestowanie systemów OCR. W trakcie realizacji projektu przeanalizowano technologie dostępnych rozwiązań oraz dokonano wyboru reprezentatywnych narzędzi: Azure AI Document Intelligence, Tesseract OCR i EasyOCR. Następnie przygotowano generator faktur, który opierał się na syntetycznych danych ze stworzonego własnego słownika. Kolejnym etapem było opracowanie i implementacja mikroservisu typu REST API integrującego wybrane silniki. Rezultatem prac były wyniki jakościowe oraz wydajnościowe ekstrakcji danych. Na ich podstawie sformułowano rekomendacje dotyczące badanych narzędzi. Rozwiązanie Tesseract okazało się najszybszym narzędziem, podczas gdy EasyOCR natrafił na wysoką barierę wydajnościową oraz wydłużony czas pracy w przypadku braku wykorzystania jednostki GPU, więc narzuca użytkownikowi warunek dysponowania sprzętem wyposażonym w procesory graficzne. Tesseract działał na zasadzie „skanera” pionowego, czytywał dane kolumna po kolumnie, dzięki czemu wypadł najlepiej w przypadku dopasowania danych z tabel w układach clean. Z kolei EasyOCR priorytetyzuje czytanie poziome, co wpłynęło na lepsze dopasowania kontekstowe, jednak problematyczne okazały się dane w postaci tabel. Oba silniki wymagają autorskich parserów, co jest z kolei niewymagane w przypadku Azure, jednakże istotnym ograniczeniem okazało się niedopasowanie usługi Microsoft do polskiego prawa podatkowego, gdzie występowały błędy w przypisywaniu odpowiednich wartości netto i brutto. Systemy lokalne nie zostały obciążone kosztami subskrypcyjnymi, jednakże wymagają dużego nakładu pracy ze strony programistycznej, zaś chmurowe rozwiązania napotykały na problemy związane z opóźnieniami sieciowymi oraz ograniczeniami elastyczności.

Warto napomnieć, iż w ostatnich miesiącach wprowadzony został Krajowy System e-Faktur, który rozwiązuje problem niejednorodnych layoutów, gdyż zostały one ogólnie ustandaryzowane w formacie XML, co usuwa problem tworzenia dodatkowych autorskich parserów. Niniejsza praca oraz wykonany projekt może zatem stanowić bazę analityczną dla proponowanych rozwiązań, a przeprowadzone badania oraz wynikające z nich wnioski stać się przyczynkiem do prac nad ogólnym systemem rozpoznawania danych ze zdjęć czy skanów faktur na poziomie krajowego zastosowania.

Bibliografia

- [1] S. Jordan, S. Sternad Zabukovšek i I. Šišovska Klančnik, „Document Management System – A Way to Digital Transformation,” *Naše Gospodarstvo / Our Economy* 68(2), p. 43–54, 2022.
- [2] Y. Xu, M. Li, L. Cui, S. Huang, F. Wei i M. Zhou, „LayoutLM: Pre-training of Text and Layout for Document Image Understanding,” w *Proceedings of the 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, Online, 2020.
- [3] P. Yusuf, S. A. Hannan, A. M. A. A. Mir i A. Mane, „An Overview and Applications of Optical Character Recognition,” *International Journal of Computer Trends and Technology*, p. 81–90, 2023.
- [4] H. Karez i K. Mehmet, „A Detailed Analysis of Optical Character Recognition Technology,” *International Journal of Applied Mathematics, Electronics and Computers*, p. 244–249, 2016.
- [5] O. Nobuyuki, „A Threshold Selection Method from Gray-Level Histograms,” *IEEE Transactions on Systems, Man, and Cybernetics*, p. 62–66, January 1979.
- [6] M. Junaid, S. Mamoona, K. R. Ahmed i U. Mueen, „Handwritten Optical Character Recognition (OCR): A Comprehensive Systematic Literature Review (SLR),” *IEEE Access*, 2020.
- [7] R. Smith, „An Overview of the Tesseract OCR Engine,” w *Proceedings of the 9th International Conference on Document Analysis and Recognition (ICDAR)*, Curitiba, Brazil, 2007.
- [8] Tesseract OCR, „Tesseract API Reference – Version 4.0.0,” Tesseract OCR Project, 2018. [Online]. Available: <https://tesseract-ocr.github.io/tessapi/4.0.0/>.
- [9] E. Sabir, S. Rawls i P. Natarajan, „Implicit Language Model in LSTM for OCR,” w *2017 14th IAPR International Conference on Document Analysis and Recognition (ICDAR)*, Kyoto, Japonia, 2017.
- [10] A. Jaied, „EasyOCR: Ready-to-use OCR with 80+ supported languages based on PyTorch,” 2020. [Online]. Available: <https://github.com/JaiedAI/EasyOCR>.
- [11] S. Raschka, J. Patterson i C. Nolet, „Machine Learning in Python: Main developments and technology trends in data science, machine learning, and artificial intelligence,” *Information* 11(4), p. 193, 2020.

- [12] M. Lathkar, High-Performance Web Apps with FastAPI: The Asynchronous Web Framework Based on Modern Python, Apress, 2023.
- [13] N. contributors, „Node.js v22.x Documentation,” 2024. [Online]. Available: <https://nodejs.org/docs/latest/api/>.
- [14] T. G. contributors, „The Go Programming Language Documentation,” 2024. [Online]. Available: <https://go.dev/doc/>.
- [15] J. A. Kreibich, Using SQLite, O'reilly Media Inc. USA, 2010.
- [16] A. Aylton, X. Laerte i V. M. Tulio, „Automatic Library Migration Using Large Language Models: First Results,” w *International Symposium on Empirical Software Engineering and Measurement*, Barcelona, 2024.
- [17] G. Leifert, T. Strauß, T. Grüning i R. Labahn, „End-to-End Measure for Text Recognition,” w *019 International Conference on Document Analysis and Recognition (ICDAR)*, Sydney, NSW, Australia, 2019.
- [18] K. Kuhn, V. Kersken i G. Zimmermann, „Beyond Levenshtein: Leveraging Multiple Algorithms for Robust Word Error Rate Computations And Granular Error Classifications,” *Interspeech*, 2024.
- [19] A. Vanacore, M. S. Pellegrino i A. Ciardiello, „Fair evaluation of classifier predictive performance based on binary confusion matrix,” *Computational Statistics* 39(6), 2022.
- [20] M. Arora, U. Kanjilal i D. Varshney, „Evaluation of information retrieval: precision and recall,” *International Journal of Indian Culture and Business Management* 12(2), p. 224, 2016.
- [21] W. Iqbal, M. N. Dailey, D. Carrera i P. Janecek, „Adaptive resource provisioning for read intensive multi-tier applications in the cloud,” *Future Generation Computer Systems* 27(6), pp. 871-879, 2011.
- [22] E. Alpaydin, Introduction to Machine Learning, Cambridge: MIT Press, 2020.
- [23] O. C. Oyeniran, A. O. Adewusi, A. G. Adeleke, L. A. Akwawa i C. F. Azubuko, „Microservices architecture in cloud-native applications: Design patterns and scalability,” *Computer Science & IT Research Journal* 5(9), pp. 2107-2124, 2024.
- [24] K. Zen, S. Mohanan, S. Tarmizi, N. Annuar i N. U. Sama, „Latency Analysis of Cloud Infrastructure for Time-Critical IoT Use Cases,” w *2022 Applied Informatics International Conference (AiIC)*, Serdang, Malezja, 2022.

- [25] Microsoft, „Read model OCR data extraction - Document Intelligence,” 2024. [Online]. Available: <https://learn.microsoft.com/en-us/azure/ai-services/document-intelligence/prebuilt/read?view=doc-intel-4.0.0&tabs=sample-code>.
- [26] P. Mell i T. Grance, „The NIST Definition of Cloud Computing,” Special Publication (NIST SP), National Institute of Standards and Technology, Gaithersburg, MD, USA, 2011.
- [27] C. Ghezzi, M. Jazayeri i D. Mandrioli, Fundamentals of software engineering (2. ed.), Mediolan: Prentice Hall, 2003.
- [28] A. Graves, S. Fernandez, F. Gomez i J. Schmidhuber, „Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks,” w *Proceedings of the Twenty-Third International Conference (ICML 2006)*, Pittsburgh, Pennsylvania, USA, 25-29 czerwca 2006.
- [29] S. Hoffstaetter, „pytesseract (Version 0.3.13),” Python, 16 sierpień 2024. [Online]. Available: <https://pypi.org/project/pytesseract/>.
- [30] D. Jurafsky i J. H. Martin, Speech and Language Processing. An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition (3rd ed. draft), Upper Saddle River, New Jersey: Prentice Hall, 2024.
- [31] T. F. Roy, *Architectural Styles and the Design of Network-based Software Architectures*, Irvine: University of California, 2000.
- [32] M. Fowler, Patterns of Enterprise Application Architecture, Boston, MA, USA: Addison Wesley, 2002.
- [33] G. von Krogh i E. von Hippel, „Open Source Software and the Private-Collective Innovation Model: Issues for Organization Science,” *Organization Science* 14(2), pp. 208-223, 2003.
- [34] Y. LeCun, Y. Bengio i G. Hinton, „Deep learning,” *Nature* 521(7553), pp. 436-44, 2015.
- [35] M. I. Jordan i T. M. Mitchell, „Machine learning: Trends, perspectives, and prospects,” *Science* 349(6245), pp. 255-60, 2015.
- [36] T. H. Davenport, „Putting the Enterprise into the Enterprise System,” *Harvard Business Review*, vol. 78, issue 4, pp. 121-131, 1998.

[37] Z. Rosiek, „Mapowanie obiektowo-relacyjne ORM - czy tylko dobra idea,” Zeszyty Naukowe Warszawskiej Wyższej Szkoły Informatyki, nr 4, pp. 99-112, 2010.

Spis rysunków

Rys. 1. Struktura projektu.....	16
Rys. 2. Diagram bazy danych.....	18
Rys. 3. Schemat działania skryptu.	20
Rys. 4. Interfejs endpointu /ocr (FastAPI).....	22
Rys. 5. Wizualna reprezentacja działania parserów.	25
Rys. 6. Blokowa reprezentacja algorytmu analizy przestrzennej.	26
Rys. 7. Fragment pliku benchmark_report.txt.	31
Rys. 8. Przykładowe faktury o 8 różnych layoutach.	32
Rys. 9. Wyniki F1-score dla silnika PyTesseract dla obrazów typu photo.....	36
Rys. 10. Porównanie wartości y dla dwóch rozpoznanych słów z jednego wiersza.....	37
Rys. 11. Fragment rozpatrywanej faktury.	38

Spis wzorów

Równanie 1. Precyzja.....	11
Równanie 2. Czułość.	11
Równanie 3. F1-score.	12

Spis kodów

Listing 1. Plik data/contractors.json.	Błąd! Nie zdefiniowano zakładki.
Listing 2. Funkcja _extract_buyer_nip	27
Listing 3. Fragment kodu azure_adapter.py.....	37
Listing 4. Fragment pliku all_results.json z błędnie przypisanymi wartościami, silnik Azure.	38

Spis tabel

Tab. 1. Wyniki duration_ms dla kolejnych silników.....	33
Tab. 2. Dokładność pól faktury.....	34
Tab. 3. Dokładność pozycji towarowych i wartości liczbowych.....	35

Abstract

The aim of this engineering thesis was to design, implement, and evaluate a system for automatic recognition and extraction of data from VAT invoices. The main research objective was to conduct an objective comparison between a custom OCR microservice, built using the free EasyOCR and PyTesseract libraries, and the commercial cloud solution Azure AI Document Intelligence.

As part of the practical component, a REST API application was developed in Python. It processed input document images using the selected OCR engine, then performed data structuring and stored the results in a relational SQL database. To ensure a repeatable and reliable testing environment, a dedicated data generator was created. Based on eight diverse HTML templates, a dataset of synthetic invoices was produced and subsequently transformed into digital documents, flat scans, and distorted mobile photographs.

In accordance with the assumptions, each of the examined technologies was tested on an extensive set of documents. Using dedicated analytical scripts, an automated evaluation process was carried out, including a detailed analysis of three key parameters: the effectiveness of data extraction and field assignment (e.g., invoice numbers, dates, contractors, line items, net and gross amounts), the accuracy of the recognized text, and the processing time of individual documents.

The completed project and the generated result summaries enabled a comprehensive comparison of the evaluated systems.